Table of Contents

Main Report

- **Credit Risk Intelligence System** *(p. 1)*
    - Big Data Driven Credit Score Classification using Hadoop Ecosystem and Python Analytics *(p. 1)*

Chapter 1: Introduction *(p. 1)*

- **1.1** About the Project *(p. 1)*
- **1.2** Problem Statement *(p. 1)*
- **1.3** Objectives *(p. 2)*
- **1.4** Methodology — Steps to Be Followed *(p. 2)*
- **1.5** Key Performance Indicators (KPIs) *(p. 2)*
- **1.6** Analytical & Visualization Plan *(p. 3)*
- **1.7** Expected Outcome *(p. 4)*

Chapter 2: System Setup and Hadoop Ecosystem Configuration *(p. 5)*

- **2.1** Introduction *(p. 5)*
- **2.2** Hardware and Software Environment *(p. 5)*
- **2.3** Setup Methodology *(p. 6)*
    - **2.3.1** System Preparation *(p. 6)*
    - **2.3.2** Hadoop Installation *(p. 6)*
    - **2.3.3** HIVE Installation *(p. 6)*
- **2.4** Memory Optimization for 4 GB RAM Constraint *(p. 8)*
- **2.5** Technical Challenges Encountered and Resolved *(p. 9)*
    - **2.5.1** Java Version Incompatibility with Hive CLI *(p. 9)*
    - **2.5.2** Corrupted Archive During Download *(p. 9)*
    - **2.5.3** Directory Structure Conflict During Installation *(p. 9)*
- **2.6** Verification of Setup *(p. 10)*
- **2.7** Significance to the Project *(p. 10)*

Chapter 3: Dataset Description *(p. 11)*

- **3.1** Dataset Overview *(p. 11)*
- **3.2** Data Dictionary *(p. 11)*

- **3.3** Known Data Quality Issues *(p. 12)*
- **3.4** Relevance to Project Objectives *(p. 13)*

Chapter 4: Data Ingestion (HDFS to HIVE) *(p. 14)*

- **4.1** Purpose of This Chapter *(p. 14)*
- **4.2** Uploading the Dataset to HDFS *(p. 14)*
- **4.3** Creating a HIVE Table Over the HDFS Data *(p. 14)*
- **4.4** Issue Encountered: Column Misalignment Due to Embedded Commas *(p. 15)*
- **4.5** Issue Encountered: MapReduce Job Failure (Missing YARN Classpath) *(p. 16)*
- **4.6** Verification of Successful Ingestion *(p. 17)*
- **4.7** Significance *(p. 17)*

Chapter 5: HIVE JDBC Connectivity — Bridging Hadoop and Python *(p. 18)*

- **5.1** Purpose of This Chapter *(p. 18)*
- **5.2** Architecture of the Connection *(p. 18)*
- **5.3** Environment Setup *(p. 18)*
- **5.4** Issue Encountered: Metastore Version Mismatch Across Directories *(p. 19)*
- **5.5** Issue Encountered: User Impersonation Authorization Error *(p. 19)*
- **5.6** Establishing the Connection from Python *(p. 20)*
- **5.7** Verification of Successful Connectivity *(p. 20)*
- **5.8** Significance *(p. 21)*

Chapter 6: Data Cleaning and Pre-processing *(p. 22)*

- **6.1** Purpose of This Chapter *(p. 22)*
- **6.2** Local Persistence of Ingested Data *(p. 22)*
- **6.3** Diagnostic Assessment of Raw Data *(p. 22)*
- **6.4** Column Name Standardization *(p. 23)*
- **6.5** Data Type Conversion *(p. 23)*
- **6.6** Verification of Missing Values Post-Conversion *(p. 24)*
- **6.7** Range and Anomaly Verification *(p. 24)*
- **6.8** Parsing the Multi-Valued type_of_loan Field *(p. 24)*
- **6.9** Removal of Sensitive and Redundant Columns *(p. 25)*

- **6.10** Categorical Value Inspection *(p. 25)*
- **6.11** Currency Conversion *(p. 26)*
- **6.12** Final Cleaned Dataset *(p. 27)*
- **6.13** Significance *(p. 28)*

Chapter 7: Exploratory Data Analysis and KPI Computation *(p. 28)*

- **7.1** Purpose of This Chapter *(p. 28)*
    - **7.1.1** What is a KPI? *(p. 29)*
- **7.2** Loading the Cleaned Dataset *(p. 29)*
- **7.3** KPI Group 1: Customer Demographics *(p. 29)*
- **7.4** KPI Group 2: Financial Behavior *(p. 30)*
- **7.5** KPI Group 3: Loan & Credit Performance *(p. 31)*
- **7.6** Feature Engineering & Ratio Analysis *(p. 32)*
- **7.7** KPI Group 4: Risk & Fraud Indicators *(p. 33)*
- **7.8** KPI Group 5: Behavioral Insights *(p. 34)*
- **7.9** Summary of Computed KPIs *(p. 35)*
- **7.10** Significance *(p. 36)*

Chapter 8: Data Visualization *(p. 36)*

- **8.1** Purpose of This Chapter *(p. 36)*
- **8.2** Demographic Analysis *(p. 37)*
- **8.3** Income & Salary Insights *(p. 37)*
- **8.4** Credit & Loan Analysis *(p. 40)*
- **8.5** Risk & Fraud Indicators *(p. 42)*
- **8.6** Behavioral Patterns *(p. 45)*
- **8.7** Overall Credit Score Analysis *(p. 47)*
- **8.8** Summary of Key Visual Findings *(p. 49)*
- **8.9** Significance *(p. 49)*

Chapter 9: Model Building & Classification *(p. 50)*

- **9.1** Purpose of This Chapter *(p. 50)*
- **9.2** Feature Selection *(p. 50)*
- **9.3** Categorical Encoding *(p. 51)*

- **9.4** Initial Modeling Attempt — Row-Level Train-Test Split *(p. 52)*
- **9.5** Diagnosing and Fixing Data Leakage *(p. 52)*
- **9.6** Model Training — Random Forest Classifier (Corrected Split) *(p. 53)*
    - **9.6.1** Baseline Comparison — Logistic Regression (Corrected Split) *(p. 54)*
    - **9.6.2** Model Comparison — Random Forest vs Logistic Regression *(p. 55)*
- **9.7** Model Evaluation (Corrected Split) *(p. 56)*
- **9.8** Confusion Matrix (Corrected Split) *(p. 57)*
- **9.9** Feature Importance *(p. 58)*
- **9.10** Model Summary and Conclusion *(p. 59)*
- **9.11** Significance *(p. 59)*

Chapter 10: Conclusion & Recommendations *(p. 60)*

- **10.1** Project Summary *(p. 60)*
- **10.2** Key Findings *(p. 60)*
- **10.3** Business Recommendations *(p. 61)*
- **10.4** Limitations *(p. 61)*
- **10.5** Future Scope *(p. 62)*
- **10.6** Final Remarks *(p. 62)*

Credit Risk Intelligence System

Big Data Driven Credit Score Classification using Hadoop Ecosystem and Python Analytics

Chapter 1: Introduction

1.1 About the Project

This project undertakes a comprehensive analysis of a real-world banking and financial services dataset, encompassing customer demographic, financial, and credit behavior attributes. The primary aim is to **detect risk and fraud-indicative patterns, evaluate customer credit risk, and derive a deeper understanding of customer financial behavior** through data-driven analysis.

The project employs a structured Data Science pipeline — including data cleaning, data wrangling, exploratory data analysis (EDA), Key Performance Indicator (KPI) computation, and multi-dimensional visualization — layered on top of a Big Data infrastructure (Hadoop, HIVE) capable of scalably storing and processing large volumes of financial records. The end objective is to derive **actionable insights** that can support real business decisions, such as loan approval policies, fraud prevention strategies, and customer risk segmentation.

1.2 Problem Statement

Financial institutions manage large volumes of customer financial data, yet extracting meaningful, decision-ready insights from this data — such as who is likely to default, which behaviors indicate elevated risk, or how credit health varies across customer segments — remains a persistent challenge due to data volume, inconsistency, and the limitations of manual evaluation.

Traditional and legacy credit evaluation systems face three key limitations:

1. **Scalability** — processing large volumes of customer financial records is slow using conventional tools.
2. **Consistency** — manual risk categorization can vary between evaluators.
3. **Delayed insights** — traditional systems struggle to quickly surface patterns (e.g., which factors most strongly predict default risk) across large populations.

This project addresses this gap by building an end-to-end analytics pipeline — leveraging the Hadoop ecosystem for scalable data storage and processing, combined with Python-based data science techniques — that transforms raw financial records into clear, quantified, and visualized insights, and automatically categorizes customers into credit risk segments (Good / Standard / Poor).

1.3 Objectives

The project is designed to achieve the following objectives:

1. Ingest and process a large-scale banking dataset using a Hadoop-based Big Data pipeline.
2. Clean and prepare the dataset to ensure analytical accuracy and consistency.
3. Define and compute a set of business-relevant Key Performance Indicators (KPIs).
4. Perform exploratory data analysis across demographic, financial, credit, and behavioral dimensions.
5. Visualize patterns and relationships using appropriate statistical charts.
6. Classify customers into credit risk categories (Good / Standard / Poor) using data science techniques.
7. Translate analytical findings into actionable business recommendations.

1.4 Methodology — Steps to Be Followed

The project follows a systematic, stage-wise methodology:

Stage	Activity
1	Load dataset into the Hadoop ecosystem (HDFS to HIVE) and into Python (via HIVE JDBC)
2	Clean dataset — correct data types, handle null values and duplicates, resolve inconsistent entries
3	Perform data wrangling to bring the dataset into an analysis-ready structure
4	Generate descriptive statistics and compute defined KPIs
5	Create visualizations across demographic, financial, credit, risk, and behavioral dimensions
6	Analyze trends, correlations, and patterns; interpret findings in business terms
7	Build a classification model to predict credit risk category
8	Conclude findings and propose data-driven recommendations aligned with business objectives

1.5 Key Performance Indicators (KPIs)

To ground the analysis in measurable, business-relevant terms, the following KPIs are defined across five analytical dimensions:

a) Customer Demographics

- Average customer age

- Distribution of customers by occupation
- Number of customers by income group

b) Financial Behavior

- Average annual income and average monthly in-hand salary
- Average Debt-to-Income Ratio
- Average Credit Utilization Ratio
- Average number of bank accounts and credit cards held

c) Loan & Credit Performance

- Average number of loans per customer
- Average interest rate
- EMI Burden Ratio (Total EMI as a percentage of income)
- Average Credit History Age

d) Risk & Fraud Indicators

- Average number of delayed payments
- Delay from due date (average, minimum, maximum)
- Percentage of customers with frequent credit inquiries
- Percentage of customers with changed credit limits
- Distribution of customers across Credit Score categories (Good / Standard / Poor)

e) Behavioral Insights

- Distribution of payment behavior patterns
- Percentage of customers who pay only the minimum amount due

1.6 Analytical & Visualization Plan

The visual analysis is organized into six thematic groups, each targeting a specific business question:

Group 1 — Demographic Analysis

- Bar Chart: Customer distribution by occupation
- Histogram: Age distribution
- Boxplot: Annual income by occupation

Group 2 — Income & Salary Insights

- Histogram: Annual income distribution
- Boxplot: Monthly in-hand salary vs. Credit Score
- KDE Plot: Income distribution comparison (Good vs. Poor credit customers)

Group 3 — Credit & Loan Analysis

- Bar Chart: Average number of loans vs. Credit Score
- Stacked Bar Chart: Types of loans availed by customers
- Scatter Plot: Outstanding debt vs. annual income (colored by Credit Score)

Group 4 — Risk & Fraud Indicators

- Boxplot: Delayed payments vs. Credit Score
- Histogram: Distribution of delay from due date
- Bar Chart: Credit Utilization Ratio vs. Credit Score
- Heatmap: Correlation among numerical features (income, debt, EMI, etc.)

Group 5 — Behavioral Patterns

- Pie Chart: Payment behavior distribution
- Bar Chart: Minimum-amount-only payment (Yes/No) vs. Credit Score
- Stacked Bar Chart: Payment behavior vs. occupation

Group 6 — Overall Credit Score Analysis

- Pie Chart: Percentage of customers in Good / Standard / Poor categories
- Bar Chart: Average debt, EMI, and utilization across Credit Score categories

1.7 Expected Outcome

By the end of this analysis, the project will produce a quantified, visualized understanding of customer credit risk drivers — enabling data-backed recommendations for loan approval criteria, early fraud/risk detection signals, and customer segmentation strategies, while simultaneously demonstrating an end-to-end Big Data and Data Science pipeline from raw data ingestion through to business insight.

Chapter 2: System Setup and Hadoop Ecosystem Configuration

2.1 Introduction

Before implementation of the Credit Risk Intelligence System could begin, it was necessary to establish a fully functional Big Data processing environment. Rather than relying on a pre-configured cloud sandbox or a virtual machine image, a **complete Hadoop ecosystem was configured from scratch on a single physical machine**, running in **pseudo-distributed mode**. This approach was deliberately chosen to gain first-hand, practical understanding of how the Hadoop Distributed File System (HDFS), YARN resource management layer, and HIVE data warehouse function together — knowledge that is foundational to the Big Data Analytics specialization of this course.

This chapter documents the environment in which the system was built, the installation and configuration process, the engineering decisions made to accommodate hardware constraints, and the real technical issues encountered and resolved during setup. This process itself forms an integral part of the project, demonstrating applied competency in Hadoop Framework fundamentals prior to any data analysis work being performed.

2.2 Hardware and Software Environment

Specification	**Detail**
Machine Type	Single personal laptop (no cluster/cloud VM used)
RAM	4 GB (physical)
Operating System	Ubuntu 24.04.4 LTS
Deployment Mode	Hadoop Pseudo-Distributed Mode (Single-Node Cluster)
Java Runtime	OpenJDK 8
Hadoop Version	Apache Hadoop 3.3.6
HIVE Version	Apache Hive 3.1.3
Metastore	Apache Derby (embedded)

Justification for Pseudo-Distributed Mode: Given the resource constraints of a 4 GB RAM single machine, a true multi-node cluster was not feasible. Pseudo-distributed mode was selected as it runs all Hadoop daemons (NameNode, DataNode, ResourceManager, NodeManager, SecondaryNameNode) as separate JVM processes on one machine, communicating over the network stack exactly as they would in a real distributed cluster. This preserves the architectural and operational realism of Hadoop while remaining feasible on constrained hardware — making it the industry-standard approach for Hadoop development and learning environments.

2.3 Setup Methodology

The setup was carried out in the following sequence:

2.3.1 System Preparation

- Verified available system resources (`free -h`, `df -h`)
- Configured a **4 GB swap file** on disk to extend virtual memory headroom beyond the physical 4 GB RAM, preventing daemon crashes under memory pressure
- Installed and configured **OpenJDK 8**, selected specifically for maximum compatibility across the Hadoop and Hive stack (see Section 5.1)
- Configured **passwordless SSH to localhost**, a mandatory requirement since Hadoop internally uses SSH to manage its daemons even in single-node mode

2.3.2 Hadoop Installation

- Downloaded and extracted Apache Hadoop 3.3.6 to `/opt/hadoop`
- Set environment variables (`HADOOP_HOME`, `HADOOP_CONF_DIR`, `PATH`) for system-wide command access
- Configured core Hadoop XML files:
    - `core-site.xml` — defined HDFS NameNode address (`hdfs://localhost:9000`)
    - `hdfs-site.xml` — set replication factor to **1** (appropriate for single-node; **default of 3 would be meaningless and wasteful of disk space on one machine)**
    - `mapred-site.xml` and `yarn-site.xml` — memory allocation tuned specifically for the 4 GB constraint (detailed below)

2.3.3 HIVE Installation

- Downloaded and extracted Apache Hive 3.1.3 to `/opt/hive`
- Set `HIVE_HOME` and updated `PATH`
- Created required HDFS directories (`/tmp`, `/user/hive/warehouse`) with appropriate write permissions for the Hive warehouse
- Initialized the Hive Metastore schema using Derby (`schematool -dbType derby -initSchema`)
- Verified functionality via the Hive CLI with `CREATE TABLE`, `SHOW TABLES`, and `DROP TABLE` operations

Execution Output — Java Installation Verification:

```
$ java -version
openjdk version "11.0.31" 2026-04-21
OpenJDK Runtime Environment (build 11.0.31+11-post-1ubuntu1-24.04.2-
Ubuntu)
OpenJDK 64-Bit Server VM (build 11.0.31+11-post-1ubuntu1-24.04.2-
Ubuntu, mixed mode, sharing)

$ javac -version
javac 11.0.31
```

(Java 11 was used initially for Hadoop; subsequently switched to Java 8 across the stack — see Section 5.1)

Execution Output — Hadoop Installation Verification:

```
$ hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r
1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled with protoc 3.7.1
This command was run using /opt/hadoop/share/hadoop/common/hadoop-
common-3.3.6.jar
```

Execution Output — Passwordless SSH Verification:

```
$ ssh localhost
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-87-generic x86_64)
Last login: Sun Apr 26 11:16:37 2026
```

(Successful login without a password prompt confirms passwordless SSH was correctly configured.)

Execution Output — NameNode Formatting:

```
$ hdfs namenode -format
...
INFO common.Storage: Storage directory /opt/hadoop/tmpdata/namenode
has been successfully formatted.
...
SHUTDOWN_MSG: Shutting down NameNode at mayankguptaji/192.168.254.232
```

Execution Output — Starting HDFS and YARN Daemons:

```
$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [mayankguptaji]

$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
```

Execution Output — Hive CLI Functional Test:

```
hive> SHOW DATABASES;
OK
default
Time taken: 1.844 seconds, Fetched: 1 row(s)

hive> CREATE TABLE test_table (id INT, name STRING);
OK
Time taken: 2.61 seconds

hive> SHOW TABLES;
OK
test_table
Time taken: 0.129 seconds, Fetched: 1 row(s)

hive> DROP TABLE test_table;
OK
Time taken: 3.41 seconds
```

2.4 Memory Optimization for 4 GB RAM Constraint

A key engineering challenge of this setup was configuring Hadoop's default memory allocations — which assume server-class hardware with 8GB+ RAM — down to values that would run reliably within a 4 GB consumer laptop. The following adjustments were made in `mapred-site.xml` and `yarn-site.xml`:

Parameter	Default	Configured Value	Reason
`mapreduce.map.memory.mb`	1024 MB	512 MB	Reduce per-task memory footprint
`mapreduce.reduce.memory.mb`	1024 MB	512 MB	Reduce per-task memory footprint
`mapreduce.map.java.opts`	-Xmx800m	-Xmx410m	Keep JVM heap safely under container limit
`yarn.nodemanager.resource.memory-mb`	8192 MB	2048 MB	Cap total YARN memory usage, reserving RAM for OS and other services
`yarn.nodemanager.vmem-check-enabled`	true	false	Prevent YARN from prematurely killing tasks due to virtual memory checks, a known false-positive issue on low-RAM systems

This tuning ensured that HDFS, YARN, and HIVE could run concurrently without triggering out-of-memory failures, while leaving sufficient headroom for the OS and the Python/Jupyter environment used in later phases of the project.

2.5 Technical Challenges Encountered and Resolved

Documenting these issues demonstrates the practical, non-trivial nature of the environment setup and the debugging skills applied.

2.5.1 Java Version Incompatibility with Hive CLI

Issue: After successful installation, launching the Hive CLI (`hive`) consistently failed with:

```
java.lang.ClassCastException: class
jdk.internal.loader.ClassLoaders$AppClassLoader
cannot be cast to class java.net.URLClassLoader
```

Root Cause: Hive 3.1.3's session initialization code relies on a Java classloader casting technique that is incompatible with Java 9+'s revised module system. The environment had initially been configured with Java 11 (installed for general Hadoop compatibility), which broke Hive specifically.

Resolution: Standardized the entire stack — both Hadoop and Hive — onto **Java 8**, which remains the most stable and widely tested JVM version for the Hadoop 3.x / Hive 3.x ecosystem. `JAVA_HOME` was updated in both `hadoop-env.sh` and the system `.bashrc`, and all daemons were restarted. This resolved the issue permanently, since Hive internally invokes the `hadoop` command, which would otherwise re-introduce Java 11 into the execution path regardless of Hive's own settings.

2.5.2 Corrupted Archive During Download

Issue: Initial extraction of the Hive tarball failed with `tar: Unexpected EOF in archive`, due to an incomplete download over an unstable network connection.

Resolution: Used `wget -c` (resume capability) to complete the download, and verified file integrity via file size comparison before re-attempting extraction — avoiding repeated full downloads and ensuring the archive was complete before use.

2.5.3 Directory Structure Conflict During Installation

Issue: A duplicate/partial extraction attempt had already populated the target installation directory (`/opt/hive`), causing a nested folder structure and a "Directory not empty" error during the final move operation.

Resolution: Performed a clean removal of the conflicting directory and re-executed the move operation, then verified the final structure contained Hive's `bin`, `conf`, and `lib` directories directly at the expected path — a good practice reinforced for all subsequent installation steps.

2.6 Verification of Setup

The completed environment was verified using the `jps` (Java Virtual Machine Process Status) utility, confirming all required daemons were active:

Execution Output — `jps` (all 5 required daemons running):

```
$ jps
3584 NameNode
3708 DataNode
3889 SecondaryNameNode
4172 ResourceManager
4280 NodeManager
4459 Jps
```

This confirms that HDFS (NameNode, DataNode, SecondaryNameNode) and YARN (ResourceManager, NodeManager) were both fully operational simultaneously on the 4 GB RAM machine, validating that the memory tuning described in Section 4 was effective.

Hive functionality was independently verified through the Hive CLI by creating, listing, and dropping a test table (output shown in Section 3.3), confirming that the Metastore, HDFS integration, and query execution engine were all operating correctly end-to-end.

(Insert screenshots here: HDFS NameNode Web UI at localhost:9870, and YARN ResourceManager Web UI at localhost:8088)

2.7 Significance to the Project

This self-configured environment forms the operational foundation for all subsequent Big Data processing in this project. Every dataset ingestion, MapReduce transformation, and HIVE query performed in later chapters runs on this independently-built infrastructure — rather than a pre-packaged sandbox — reflecting genuine applied understanding of the Hadoop Framework, its daemon architecture, and real-world resource-constrained deployment considerations, consistent with the learning outcomes.

Chapter 3: Dataset Description

3.1 Dataset Overview

The dataset used in this project is a banking and financial services dataset containing customer-level records related to demographics, income, credit accounts, loan history, payment behavior, and an assigned credit score category. It closely resembles real-world data maintained by financial institutions and credit bureaus for credit risk assessment purposes.

- **Source:** Credit Score Classification dataset from Kaggle
- **Number of Columns:** 28
- **Target Variable:** `Credit_Score` (categorical: Good / Standard / Poor)
- **Nature of Data:** Mixed — numerical, categorical, and semi-structured (multi-value) fields

3.2 Data Dictionary

Column Name	Description	Data Type
ID	Unique record identifier	String/Object
Customer_ID	Unique identifier for each customer (may repeat across months)	String/Object
Month	Month of the record	String/Object
Name	Customer's name	String/Object
Age	Customer's age	Numeric (contains anomalies)
SSN	Social Security Number (sensitive identifier)	String/Object
Occupation	Customer's profession	Categorical
Annual_Income	Customer's total yearly income	Numeric
Monthly_Inhand_Salary	Monthly take-home salary	Numeric
Num_Bank_Accounts	Number of bank accounts held	Numeric
Num_Credit_Card	Number of credit cards held	Numeric
Interest_Rate	Interest rate applicable on credit/loans	Numeric
Num_of_Loan	Number of loans taken	Numeric
Type_of_Loan	Types of loans held (multi-value, comma-separated)	Categorical (multi-label)

Column Name	Description	Data Type
Delay_from_due_date	Average number of days payment was delayed	Numeric
Num_of_Delayed_Payment	Number of delayed payments	Numeric
Changed_Credit_Limit	Change in credit limit	Numeric
Num_Credit_Inquiries	Number of credit inquiries made	Numeric
Credit_Mix	Qualitative rating of credit mix (Good/Standard/Bad)	Categorical
Outstanding_Debt	Remaining unpaid debt amount	Numeric
Credit_Utilization_Ratio	Percentage of available credit being used	Numeric
Credit_History_Age	Length of credit history (e.g., "22 Years 1 Months")	String (needs conversion)
Payment_of_Min_Amount	Whether customer pays only the minimum due (Yes/No)	Categorical
Total_EMI_per_month	Total EMI obligation per month	Numeric
Amount_invested_monthly	Monthly investment amount	Numeric
Payment_Behaviour	Descriptive spending/payment pattern label	Categorical
Monthly_Balance	Remaining balance at month end	Numeric
Credit_Score	**Target variable — credit risk category**	**Categorical (Good/Standard/Poor)**

3.3 Known Data Quality Issues

A preliminary inspection of the dataset revealed several data quality issues that are common in real-world financial datasets and will be addressed in the upcoming Data Cleaning phase (Chapter 4):

1. **Anomalous Age values** — Some records contain negative or unrealistically high age values, indicating data entry or extraction errors.
2. **Non-numeric `Credit_History_Age`** — Stored as a descriptive string (e.g., "22 Years 1 Months") rather than a numeric value, requiring conversion to total months for analysis.

3. **Multi-valued `Type_of_Loan`** — A single field containing multiple comma-separated loan types, requiring parsing/splitting before it can be used in aggregation or visualization.
4. **Missing and inconsistent values** — Several numeric fields (e.g., `Monthly_Balance`, `Amount_invested_monthly`, `Monthly_Inhand_Salary`) contain missing entries or placeholder/garbage values that must be handled during cleaning.
5. **Sensitive identifiers** — Columns such as `Name` and `SSN` carry personally identifiable information and are excluded from analysis and modeling, retained only for potential record traceability.
6. **Repeated customer records across months** — Since each `Customer_ID` may appear across multiple `Month` entries, care is needed to avoid duplication bias during aggregation and KPI computation.

3.4 Relevance to Project Objectives

This dataset directly supports the project's objectives: demographic fields (`Age`, `Occupation`) enable customer segmentation, financial fields (`Annual_Income`, `Outstanding_Debt`, `Credit_Utilization_Ratio`) enable financial behavior analysis, payment and delay-related fields enable risk and fraud-pattern detection, and the `Credit_Score` target variable enables supervised classification — together forming a complete foundation for the KPIs and visualizations defined in Chapter 1.

Chapter 4: Data Ingestion (HDFS to HIVE)

4.1 Purpose of This Chapter

With the Hadoop ecosystem configured (Chapter 2) and the dataset structure understood (Chapter 3), this chapter documents the process of physically bringing the dataset into the Big Data environment — uploading the raw CSV file into HDFS and creating a structured HIVE table over it, so that the data can be queried using SQL-like syntax and later accessed from Python via HIVE JDBC connectivity.

4.2 Uploading the Dataset to HDFS

A dedicated directory was created within HDFS to organize project data:

```
hdfs dfs -mkdir -p /user/credit_project/input
```

The dataset file was then uploaded from local disk into HDFS using the `put` command:

```
hdfs dfs -put credit_data.csv /user/credit_project/input/
```

Successful upload was verified using:

```
hdfs dfs -ls /user/credit_project/input/
```

Execution Output:

```
Found 1 items
-rw-r--r--   1 mayankguptaji supergroup   30752741 2026-08-15 18:43
/user/credit_project/input/credit_data.csv
```

The file size (30,752,741 bytes, ~30 MB) exactly matched the original source file, confirming the upload completed without data loss or truncation.

Once uploaded, the file resides permanently within HDFS's distributed storage, independent of its original location on the local filesystem. All subsequent access to this data — by HIVE, MapReduce, or Python (via HIVE JDBC) — occurs directly through HDFS, referencing this stored location rather than the original local file.

4.3 Creating a HIVE Table Over the HDFS Data

A dedicated HIVE database was created to logically group all project-related tables:

```
CREATE DATABASE IF NOT EXISTS credit_project;
USE credit_project;
```

An **external table** was then defined over the uploaded CSV file. An external table was used (rather than a managed/internal table) so that the underlying data continues to reside at its HDFS location and is not moved or duplicated into HIVE's internal warehouse directory — **preserving a clear separation between raw ingested data and HIVE's metadata layer.**

4.4 Issue Encountered: Column Misalignment Due to Embedded Commas

Problem: An initial version of the table was created using a simple comma-delimited row format (`FIELDS TERMINATED BY ','`). Upon inspecting the loaded data (`SELECT * FROM credit_data LIMIT 5`), it was observed that column values were misaligned — later columns (including the target variable, `Credit_Score`) did not appear in their expected positions.

Root Cause: The `Type_of_Loan` field in the dataset contains multiple loan types as a single, comma-separated, quote-enclosed string (e.g., `"Auto Loan, Credit-Builder Loan, Personal Loan, and Home Equity Loan"`). A simple delimiter-based row format treats every comma as a column separator, regardless of whether it falls inside quotes — causing this single field to be incorrectly split into multiple columns, which in turn shifted every subsequent column out of position.

Resolution: The table was recreated using HIVE's `OpenCSVSerde` (Serializer/Deserializer), which performs proper CSV parsing — correctly treating quote-enclosed commas as part of a single field rather than as delimiters:

```
CREATE EXTERNAL TABLE credit_data (
    ID STRING, Customer_ID STRING, Month STRING, Name STRING, Age
STRING,
    SSN STRING, Occupation STRING, Annual_Income STRING,
    Monthly_Inhand_Salary STRING, Num_Bank_Accounts STRING,
    Num_Credit_Card STRING, Interest_Rate STRING, Num_of_Loan STRING,
    Type_of_Loan STRING, Delay_from_due_date STRING,
    Num_of_Delayed_Payment STRING, Changed_Credit_Limit STRING,
    Num_Credit_Inquiries STRING, Credit_Mix STRING, Outstanding_Debt
STRING,
    Credit_Utilization_Ratio STRING, Credit_History_Age STRING,
    Payment_of_Min_Amount STRING, Total_EMI_per_month STRING,
    Amount_invested_monthly STRING, Payment_Behaviour STRING,
    Monthly_Balance STRING, Credit_Score STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES (
   "separatorChar" = ",",
   "quoteChar"     = "\""
)
STORED AS TEXTFILE
LOCATION '/user/credit_project/input/'
TBLPROPERTIES ("skip.header.line.count"="1");
```

All columns were defined as `STRING` type at this stage — a deliberate design choice, since `OpenCSVSerde` returns all fields as strings, and the dataset was already known (Chapter 3) to contain messy, inconsistent numeric fields. Proper type conversion and cleaning was deferred to the Python(Pandas) preprocessing stage (Chapter 5), where greater control over data cleaning logic is available.

Following this fix, column alignment was verified as correct, with `Credit_Score` correctly appearing as the final field.

Execution Output — Corrected Query Result (`SELECT * FROM credit_data LIMIT 5`):

```
5634  3392  1  Aaron Maashoh  23.0  821000265.0  Scientist  19114.12
1824.84  3.0  4.0  3.0  4.0
Auto Loan, Credit-Builder Loan, Personal Loan, and Home Equity Loan
3.0  7.0  11.27  4.0  Good  809.98  26.82  265.0  No  49.57  21.46
High_spent_Small_value_payments  312.49  Good
```

(Row shown wrapped for readability; in the actual output all 28 fields appear on a single tab-separated line.) The `Type_of_Loan` field is now preserved as a single coherent value, and the target variable `Credit_Score` ("Good") correctly appears in the final column, along with `Payment_Behaviour` and `Monthly_Balance` in their correct positions — confirming the misalignment was fully resolved.

4.5 Issue Encountered: MapReduce Job Failure (Missing YARN Classpath)

Problem: Running an aggregate query (`SELECT COUNT(*) FROM credit_data`) — which triggers a MapReduce job — failed with the error:

```
Error: Could not find or load main class
org.apache.hadoop.mapreduce.v2.app.MRAppMaster
```

Root Cause: YARN's ApplicationMaster container could not locate the MapReduce framework's Java classes at runtime. This occurred because the `yarn.application.classpath` property, which tells YARN where to find Hadoop's core and MapReduce libraries, had not been explicitly configured — a step commonly required in manually configured single-node Hadoop setups.

Resolution: The missing classpath was added to `yarn-site.xml`, explicitly pointing to Hadoop's common, HDFS, MapReduce, and YARN library directories:

```
<property>
    <name>yarn.application.classpath</name>

<value>/opt/hadoop/etc/hadoop:/opt/hadoop/share/hadoop/common/lib/*:/
opt/hadoop/share/hadoop/common/*:/opt/hadoop/share/hadoop/hdfs/*:/
opt/hadoop/share/hadoop/hdfs/lib/*:/opt/hadoop/share/hadoop/
mapreduce/*:/opt/hadoop/share/hadoop/mapreduce/lib/*:/opt/hadoop/
share/hadoop/yarn/*:/opt/hadoop/share/hadoop/yarn/lib/*</value>
</property>
```

After updating the configuration, all Hadoop daemons were restarted (`stop-yarn.sh`, `stop-dfs.sh`, `start-dfs.sh`, `start-yarn.sh`) to apply the change. The `COUNT(*)` query was then re-executed successfully.

4.6 Verification of Successful Ingestion

The final row count query confirmed complete and correct ingestion of the dataset:

```
SELECT COUNT(*) FROM credit_data;
```

Execution Output:

```
Hadoop job information for Stage-1: number of mappers: 1; number of
reducers: 1
2026-08-15 19:19:33,818 Stage-1 map = 0%,   reduce = 0%
2026-08-15 19:20:33,734 Stage-1 map = 100%, reduce = 0%,   Cumulative
CPU 7.89 sec
2026-08-15 19:21:22,455 Stage-1 map = 100%, reduce = 100%, Cumulative
CPU 12.93 sec
MapReduce Total cumulative CPU time: 13 seconds 80 msec
Stage-Stage-1: Map: 1  Reduce: 1  Cumulative CPU: 13.08 sec  HDFS
Read: 30771097  HDFS Write: 106  SUCCESS
OK
100000
Time taken: 245.832 seconds, Fetched: 1 row(s)
```

Result: 100,000 records — matching the expected size of the source dataset, confirming that all rows were successfully parsed and loaded without loss.

The query completed in approximately 245 seconds (4 mapper/reducer stages combined) via a MapReduce job, which is consistent with expectations for a resource-constrained (4 GB RAM) single-node cluster performing a full table scan of a ~30 MB file with 100,000 rows.

4.7 Significance

This chapter demonstrates the practical realities of Big Data ingestion pipelines — where raw, real-world data rarely loads cleanly on the first attempt. The two issues resolved here (delimiter-based parsing failure on embedded commas, and YARN classpath misconfiguration) are genuine, commonly encountered problems in Hadoop/HIVE deployments, and their diagnosis and resolution reflect applied, hands-on understanding of the Hadoop Framework and HIVE data analysis components.

With the dataset now fully and correctly available as a queryable HIVE table (`credit_project.credit_data`, 100,000 records), the project is ready to proceed to Python-based data access via HIVE JDBC connectivity.

Chapter 5: HIVE JDBC Connectivity — Bridging Hadoop and Python

5.1 Purpose of This Chapter

With the dataset successfully ingested into a HIVE table (Chapter 4), this chapter establishes the critical link between the Big Data layer (Hadoop/HIVE) and the Data Science layer (Python) that this project's architecture depends on. Specifically, it documents how **HiveServer2** was configured and how **HIVE JDBC connectivity** (via the `pyhive` library) was used to pull data directly from HIVE into a Python/Pandas DataFrame — enabling all subsequent cleaning, analysis, and modeling to operate on data sourced live from the Hadoop ecosystem rather than a static local copy.

5.2 Architecture of the Connection

Unlike the Hive CLI (used in Chapter 4), which interacts with Hive directly, Python-based JDBC-style access requires a dedicated background service — **HiveServer2** — that listens for client connections (over the Thrift protocol) on a network port (default: 10000) and executes queries on behalf of connecting clients. The complete data path for this project is:

```
CSV File → HDFS Storage → HIVE External Table → HiveServer2 (port
10000)
    → Python (pyhive) → Pandas DataFrame
```

5.3 Environment Setup

The following Python libraries were installed within the project's virtual environment (`.venv`) to enable JDBC-style connectivity:

```
pip install pyhive thrift thrift-sasl pure-sasl pandas jupyter
```

Note on `pure-sasl`: The commonly referenced `sasl` package failed to build from source (`fatal error: longintrepr.h: No such file or directory`), as it relies on C internals removed in Python 3.11+. This was resolved by substituting **`pure-sasl`**, a pure-Python implementation providing equivalent authentication functionality without requiring native compilation — a practical adaptation for modern Python environments.

HiveServer2 was launched from the terminal:

```
cd ~/Desktop/project
hiveserver2
```

5.4 Issue Encountered: Metastore Version Mismatch Across Directories

Problem: On first attempting to start HiveServer2, the process appeared to loop through repeated session initializations without completing startup, and no process was found listening on port 10000 (`ss -tulnp | grep 10000` returned no output).

Root Cause: Investigation of `jps` and the Hive log file (`/tmp/<user>/hive.log`) revealed the actual exception:

```
MetaException: Version information not found in metastore.
```

This occurred because Derby (Hive's embedded metastore database) creates its `metastore_db` folder in whichever directory a Hive command is launched from. `hiveserver2` had been invoked from a different working directory than the one used earlier (`~/Desktop/project`) for schema initialization and table creation — resulting in two separate `metastore_db` folders, one correctly initialized and one empty/unversioned.

Resolution: The stray, uninitialized `metastore_db` folder was removed, and a standing practice was adopted to always launch Hive-related commands (`hive`, `hiveserver2`, `schematool`) from the same fixed working directory (`~/Desktop/project`), ensuring all components consistently reference the same metastore.

5.5 Issue Encountered: User Impersonation Authorization Error

Problem: After HiveServer2 started successfully (confirmed listening on port 10000), the first Python connection attempt failed with:

```
AuthorizationException: User: mayankguptaji is not allowed to
impersonate mayankguptaji
```

Root Cause: By default, Hadoop's security layer restricts which users a service account is permitted to "impersonate" (act on behalf of) when accessing HDFS. HiveServer2, by default, opens sessions on behalf of the connecting client's username — and without explicit authorization, HDFS's `ProxyUser` mechanism blocks this, even when the service account and the impersonated user are the same.

Resolution: Proxy user permissions were explicitly granted in `core-site.xml`:

```
<property>
    <name>hadoop.proxyuser.mayankguptaji.hosts</name>
    <value>*</value>
</property>
<property>
    <name>hadoop.proxyuser.mayankguptaji.groups</name>
    <value>*</value>
</property>
```

Secondary issue found during this fix: While editing configuration files, it was discovered that `hdfs-site.xml`, `mapred-site.xml`, and `yarn-site.xml` had each accumulated a **duplicate nested `<configuration>` tag** (from earlier edits made by appending rather than replacing content), resulting in invalid XML structure. All four configuration files were rewritten cleanly with a single top-level `<configuration>` block, verified using:

```
grep -c "<configuration>" /opt/hadoop/etc/hadoop/*.xml
```

confirming exactly one occurrence per file. Following this fix and a full restart of HDFS, YARN, and HiveServer2, the connection succeeded.

5.6 Establishing the Connection from Python

With HiveServer2 running and correctly configured, the connection was established from the Jupyter Notebook (`project.ipynb`) using `pyhive`:

```
from pyhive import hive
import pandas as pd

# Connect to HiveServer2
conn = hive.Connection(host='localhost', port=10000,
database='credit_project')

# Query the HIVE table directly into a Pandas DataFrame
query = "SELECT * FROM credit_data"
df = pd.read_sql(query, conn)

print(df.shape)
df.head()
```

5.7 Verification of Successful Connectivity

Execution Output:

```
(100000, 28)
```

The DataFrame shape confirmed **100,000 rows and 28 columns** — an exact match with the record count independently verified in HIVE (Chapter 4, Section 4.6) — confirming

that data was transferred completely and without loss from HDFS, through HIVE, over JDBC, into the Python environment.

Sample output (`df.head()`) confirmed all columns loaded correctly and in order, including the target variable `credit_data.credit_score`, with values consistent with those observed directly in the Hive CLI.

5.8 Significance

This chapter completes the technical bridge central to the project's architecture: a live, queryable connection between the Hadoop/HIVE Big Data layer and the Python Data Science environment. Three distinct, realistic infrastructure issues were diagnosed and resolved in this process — a Python packaging/build incompatibility, a Hive metastore consistency issue, and a Hadoop security (proxy user) misconfiguration compounded by invalid XML — reflecting practical, applied competency in HIVE JDBC connectivity, one of the explicit learning outcomes of this course's Big Data Analytics specialization.

With this connection in place, all subsequent chapters (Data Cleaning, EDA, Visualization, and Model Building) operate on data pulled live from the Hadoop ecosystem via this established pipeline, rather than a disconnected local file.

Chapter 6: Data Cleaning and Pre-processing

6.1 Purpose of This Chapter

With the dataset successfully retrieved from HIVE via JDBC (Chapter 5), this chapter documents the systematic diagnosis and resolution of data quality issues in preparation for exploratory data analysis and model building. Rather than assuming which issues exist based on prior knowledge of similar datasets, each cleaning decision in this chapter is grounded in a diagnostic step performed directly on the ingested data — following a "**diagnose, then resolve, then verify**" methodology consistent with the approach used for infrastructure issues in earlier chapters.

To preserve data lineage, all cleaning was performed on a working copy (`df_clean`), keeping the original ingested DataFrame (`df`) untouched for reference and before/after comparison.

6.2 Local Persistence of Ingested Data

To avoid repeated dependency on Hadoop and HiveServer2 for every analysis session, the DataFrame retrieved via JDBC was persisted locally as a CSV checkpoint immediately after ingestion:

```
df.to_csv('raw_credit_data_from_hive.csv', index=False)
```

This established a practical pattern followed throughout the cleaning process: after each major stage, a checkpoint CSV was saved, allowing work to resume instantly (without restarting Hadoop services or re-running prior steps) in the event of a kernel or system restart — a genuine operational consideration on a resource-constrained single machine.

6.3 Diagnostic Assessment of Raw Data

Before any cleaning was performed, the raw ingested DataFrame was inspected directly to identify actual (rather than assumed) data quality issues.

Execution Output — Structure Check:

```
(100000, 28)
dtypes: str(28)
memory usage: 21.4 MB
```

All 28 columns were loaded as string (`str`) type — an expected consequence of using `OpenCSVSerde` during HIVE table creation (Chapter 4), which returns all fields as text.

Execution Output — Missing Value Check (on raw string data):

```
All columns: 0 missing values
```

This result was noted as **misleading rather than reassuring** — since all columns were still string type, disguised missing values (e.g., blank strings or placeholder text) would

not be captured by `isnull()`. This is documented as a key finding: apparent "completeness" prior to type conversion cannot be trusted at face value.

Execution Output — Sample Diagnostics:

Field	Finding
`age`	43 unique values; sample: 23.0, 28.0, 34.0, 54.0, 55.0...
`credit_history_age`	Already present as numeric-looking values (e.g., 265.0, 394.0), contrary to the "X Years Y Months" text format assumed in Chapter 3 based on general knowledge of similar datasets
`type_of_loan`	6,261 unique combinations; "No Data" placeholder present in 11,408 records (~11.4%); duplicate loan types within single entries observed (e.g., "Auto Loan, Auto Loan, and Not Specified")
`customer_id`	12,500 unique customers across 100,000 rows — confirming each customer has approximately 8 monthly records

This diagnostic step corrected an important assumption: the `credit_history_age` field in this specific dataset copy was already stored as a numeric value (representing total months), not as a descriptive string requiring parsing — demonstrating the importance of verifying assumed issues against the actual data rather than relying solely on general dataset knowledge.

6.4 Column Name Standardization

Column names retrieved via HIVE carried a `credit_data.` prefix (HIVE's default naming convention when selecting from a table). This was removed for cleaner downstream code:

```
df_clean.columns = [col.replace('credit_data.', '') for col in
df_clean.columns]
```

6.5 Data Type Conversion

Seventeen columns identified as inherently numeric (age, income, debt, ratios, counts, etc.) were converted from string to proper numeric type:

```
for col in numeric_cols:
    df_clean[col] = pd.to_numeric(df_clean[col], errors='coerce')
```

The `errors='coerce'` parameter ensures that any non-numeric or malformed value is safely converted to `NaN` rather than raising an exception — simultaneously correcting

data types and exposing any previously hidden missing/garbage values.

Execution Output (verified via `df_clean.dtypes`): All 17 target columns confirmed as `float64`.

6.6 Verification of Missing Values Post-Conversion

Following numeric conversion, missing values were re-checked:

```
df_clean.isnull().sum()
```

Result: 0 missing values across all columns. Unlike the earlier (misleading) check on string data, this result is meaningful, since it was obtained after values were required to genuinely parse as numbers. This confirms the dataset — for this specific copy — did not contain literal garbage tokens (e.g., special characters) in its numeric fields, a genuinely useful finding that removed the need for missing-value imputation.

6.7 Range and Anomaly Verification

To check for values that are technically valid numbers but logically unrealistic (e.g., negative age), descriptive statistics were reviewed for key numeric fields:

Execution Output:

Field	Min	Max	Mean
age	14.0	56.0	33.3
annual_income	7,005.93	179,987.28	50,505.12
num_bank_accounts	0	11	5.4
num_credit_card	0	11	5.5
interest_rate	1.0	34.0	14.5
num_of_loan	0	9	3.5
delay_from_due_date	0	62	21.1
num_credit_inquiries	0	17	5.8

Finding: Contrary to anomalies commonly reported for similar credit score datasets (e.g., negative ages, extreme outlier values), this specific dataset copy showed **no impossible or out-of-range values** across the fields examined. This result is documented as evidence that the assumed data quality issues listed in Chapter 3 (based on general knowledge of similar datasets) do not fully apply to this specific data — reinforcing the value of direct diagnostic verification over assumption.

6.8 Parsing the Multi-Valued `type_of_loan` Field

The `type_of_loan` field stores multiple loan types per customer as a single comma-separated string (e.g., "Auto Loan, Credit-Builder Loan, and Home Equity Loan"), with

"No Data" used as a placeholder for customers holding no loans.

Step 1 — Placeholder standardization:

```
df_clean['type_of_loan'] = df_clean['type_of_loan'].replace('No Data',
'None')
```

Step 2 — Initial feature extraction (loan count):

```
df_clean['num_loan_types'] = df_clean['type_of_loan'].apply(
    lambda x: 0 if x == 'None' else len(str(x).split(','))
)
```

Issue identified: Manual inspection of sample output revealed that some entries contained **duplicate loan types within the same field** (e.g., "Debt Consolidation Loan, Debt Consolidation Loan, ..."), causing a simple comma-count to overstate the true number of distinct loan types held by a customer.

Correction — unique loan type counting:

```
def count_unique_loans(loan_str):
    if loan_str == 'None':
        return 0
    loans = [loan.strip() for loan in str(loan_str).split(',')]
    return len(set(loans))

df_clean['num_loan_types'] =
df_clean['type_of_loan'].apply(count_unique_loans)
```

Verification: Sample comparison confirmed the correction reduced counts appropriately for records with duplicate entries (e.g., one record's count changed from 7 to 5, another from 5 to 4), while records without duplicates remained unchanged — confirming the fix behaved correctly.

6.9 Removal of Sensitive and Redundant Columns

The `name` and `ssn` columns, identified in Chapter 3 as personally identifiable information (PII) with no analytical value, were removed.

```
df_clean = df_clean.drop(columns=['name', 'ssn',])
```

Result: Column count reduced from 28 to 27 (two columns removed, one derived column added).

6.10 Categorical Value Inspection

Remaining categorical columns were inspected for disguised placeholder or garbage values:

Execution Output:

```
occupation: 15 valid categories (Lawyer, Engineer, Architect, ... ),
no anomalies
credit_mix: Standard (45,848), Good (30,384), Bad (23,768)
payment_of_min_amount: Yes (52,326), No (35,667), NM (12,007)
payment_behaviour: 6 valid behavior categories, no anomalies
credit_score: Standard (53,174), Poor (28,998), Good (17,828)
```

Findings:

1. Categorical fields were found to be largely clean, with no disguised placeholder garbage (e.g., no underscore-only values commonly seen in similar datasets).
2. `payment_of_min_amount` contains a third category, **"NM" (Not Mentioned)**, alongside "Yes"/"No", present in approximately 12% of records. This was retained as a valid response category rather than treated as missing.
3. The target variable, **`credit_score`, shows class imbalance**: Standard (53.2%), Poor (29.0%), Good (17.8%). This is noted for consideration during model building (Chapter 8), where class weighting or stratified sampling may be required.

6.11 Currency Conversion

The dataset originates from a US-based source (indicated by the presence of an `ssn` field, a US-specific identifier), with monetary values denominated in USD. For contextual relevance to this project's Indian institutional setting, all monetary columns (`annual_income`, `monthly_inhand_salary`, `outstanding_debt`, `total_emi_per_month`, `amount_invested_monthly`, `monthly_balance`) were converted to Indian Rupees (INR) using an approximate exchange rate of **1 USD = ₹83**, with results rounded to 2 decimal places for standard currency precision:

```
USD_TO_INR = 83
monetary_cols = ['annual_income', 'monthly_inhand_salary',
'outstanding_debt',
                 'total_emi_per_month', 'amount_invested_monthly',
'monthly_balance']

for col in monetary_cols:
    df_clean[col] = (df_clean[col] * USD_TO_INR).round(2)
```

This conversion is applied for presentation and interpretability purposes only and does not affect the underlying statistical relationships (ratios, correlations, or distributions) within the data. This step is noted as a practical adaptation rather than a data quality correction — the underlying values were already valid; only their unit of denomination was changed.

Operational note: During implementation, this cell was inadvertently executed twice within a single kernel session, causing a compounding (double) conversion — evident when `annual_income` values appeared in the crores rather than lakhs. This was

resolved by restarting the Jupyter kernel, reloading the pre-conversion checkpoint, and reapplying the conversion exactly once. This is documented as a practical lesson in reproducible notebook-based data processing: transformation steps that scale existing values (rather than recomputing from a fixed source) are not safe to re-run accidentally, and checkpointing prior to such steps proved valuable for quick recovery.

6.12 Final Cleaned Dataset

Execution Output — Final Verification:

```
Final shape: (100000, 27)

id                          int64
customer_id                 int64
month                       int64
age                       float64
occupation                    str
annual_income             float64
monthly_inhand_salary     float64
num_bank_accounts         float64
num_credit_card           float64
interest_rate             float64
num_of_loan               float64
delay_from_due_date       float64
num_of_delayed_payment    float64
changed_credit_limit      float64
num_credit_inquiries      float64
credit_mix                    str
outstanding_debt          float64
credit_utilization_ratio  float64
type_of_loan                  str
credit_history_age        float64
payment_of_min_amount         str
total_emi_per_month       float64
amount_invested_monthly   float64
payment_behaviour             str
monthly_balance           float64
credit_score                  str
num_loan_types              int64
```

The cleaned dataset — 100,000 records across 27 columns, with correct data types, verified completeness, no PII, and a derived loan-diversity feature — was saved as a final checkpoint:

```
df_clean.to_csv('cleaned_credit_data_final.csv', index=False)
```

6.13 Significance

This chapter demonstrates a rigorous, evidence-based approach to data cleaning: rather than applying generic fixes based on assumptions about similar datasets, each transformation was preceded by a diagnostic step performed on the actual data, with findings explicitly compared against initial assumptions (Chapter 3). Notably, several commonly expected issues (negative ages, missing values, garbage placeholders) were tested for and found **not** to be present in this dataset copy — an equally valid and important finding as identifying issues that *do* exist, reflecting genuine analytical rigor rather than the automatic application of a generic cleaning checklist.

The dataset is now fully prepared — free of missing values, correctly typed, free of PII, and enriched with a derived feature — for exploratory data analysis and KPI computation.

Chapter 7: Exploratory Data Analysis and KPI Computation

7.1 Purpose of This Chapter

This chapter computes the Key Performance Indicators (KPIs) originally defined in Chapter 1 (Section 1.5), using the fully cleaned dataset produced in Chapter 6. These KPIs convert the cleaned dataset into concise, business-relevant metrics across five analytical dimensions, forming the quantitative foundation for the visualizations built in

Chapter 8.

7.1.1 What is a KPI?

A **Key Performance Indicator (KPI)** is a measurable value that quantifies how well a specific business objective is being achieved. In a business or analytics context, KPIs convert raw data into concise, interpretable metrics that decision-makers can act upon — for example, "average debt-to-income ratio" is far more actionable at a glance than the raw `outstanding_debt` and `annual_income` columns individually.

In this project, KPIs serve two purposes:

1. They summarize the cleaned dataset into meaningful, business-relevant numbers across five dimensions (Demographics, Financial Behavior, Loan & Credit Performance, Risk & Fraud Indicators, and Behavioral Insights), as originally defined in the project's methodology (Chapter 1, Section 1.5).
2. They form the analytical foundation for the visualizations built in Chapter 8, ensuring that each chart is grounded in a specific, pre-defined business question rather than being created arbitrarily.

7.2 Loading the Cleaned Dataset

```
import pandas as pd
df_clean = pd.read_csv('cleaned_credit_data_final.csv')
print(df_clean.shape)
```

Output: `(100000, 27)` — confirming the cleaned dataset from Chapter 6 loaded successfully.

7.3 KPI Group 1: Customer Demographics

This section computes demographic KPIs: average customer age, distribution of customers by occupation, and number of customers grouped by income level.

```
avg_age = df_clean['age'].mean()

occupation_dist = df_clean['occupation'].value_counts()

bins = [0, 1000000, 2000000, 3000000, 5000000, 8000000, float('inf')]
labels = ['<10L', '10L-20L', '20L-30L', '30L-50L', '50L-80L', '80L+']
df_clean['income_group'] = pd.cut(df_clean['annual_income'],
bins=bins, labels=labels)
income_group_dist =
df_clean['income_group'].value_counts().sort_index()
```

Note on income group bins: Bin boundaries were defined in INR (lakhs) rather than the originally planned USD-scale boundaries, since `annual_income` was converted to INR during cleaning (Chapter 6, Section 6.11). Bins were iteratively adjusted to produce a reasonably balanced distribution across brackets rather than concentrating the majority of customers in a single open-ended bracket.

Execution Output:

Metric	Result
Average Customer Age	**33.32 years**

Occupation Distribution (15 categories, all reasonably balanced):

Occupation	Count	Occupation	Count
Lawyer	7,096	Doctor	6,568
Engineer	6,864	Journalist	6,536
Architect	6,824	Manager	6,432
Mechanic	6,776	Musician	6,352
Scientist	6,744	Writer	6,304
Accountant	6,744		
Developer	6,720		
Media_Manager	6,720		
Teacher	6,672		
Entrepreneur	6,648		

Customers by Income Group (INR):

Income Bracket	Customers
<10L	7,960
10L–20L	24,944
20L–30L	15,888
30L–50L	18,392
50L–80L	18,944
80L+	13,872

7.4 KPI Group 2: Financial Behavior

This section computes KPIs related to customers' financial standing: average income metrics, debt-to-income ratio, credit utilization, and account/card ownership.

```
avg_annual_income = df_clean['annual_income'].mean()
avg_monthly_salary = df_clean['monthly_inhand_salary'].mean()

df_clean['debt_to_income_ratio'] = df_clean['outstanding_debt'] /
df_clean['annual_income']
avg_dti = df_clean['debt_to_income_ratio'].mean()
```

```
avg_cur = df_clean['credit_utilization_ratio'].mean()
avg_bank_accounts = df_clean['num_bank_accounts'].mean()
avg_credit_cards = df_clean['num_credit_card'].mean()
```

Execution Output:

KPI	Result
Average Annual Income	₹41,91,925.25
Average Monthly In-Hand Salary	₹3,48,373.48
Average Debt-to-Income Ratio	0.061 (6.1%)
Average Credit Utilization Ratio	32.29%
Average Number of Bank Accounts	5.37
Average Number of Credit Cards	5.53

(Note: Currency values are displayed without thousands separators in the notebook output for simplicity; the values above are formatted for report readability.)

7.5 KPI Group 3: Loan & Credit Performance

This section computes KPIs related to customers' loan and credit account performance: average number of loans, interest rates, EMI burden relative to income, and credit history length.

```
avg_num_loans = df_clean['num_of_loan'].mean()
avg_interest_rate = df_clean['interest_rate'].mean()

df_clean['emi_burden_ratio'] = (df_clean['total_emi_per_month'] /
df_clean['monthly_inhand_salary']) * 100
avg_emi_burden = df_clean['emi_burden_ratio'].mean()

avg_credit_history_months = df_clean['credit_history_age'].mean()
avg_credit_history_years = avg_credit_history_months / 12
```

Execution Output:

KPI	Result
Average Number of Loans per Customer	3.53
Average Interest Rate	14.53%
Average EMI Burden Ratio	2.98% of monthly income
Average Credit History Age	221.2 months (~18.4 years)

Insight: The relatively low average EMI burden ratio (2.98%) suggests that, on average, customers in this dataset are not heavily loan-repayment-constrained relative to their income — a finding that will be further examined against credit score category in Chapter 8.

7.6 Feature Engineering & Ratio Analysis

To support both deeper analysis and improved model accuracy in later chapters, three domain-specific financial ratios were derived from the cleaned dataset. These ratios consolidate raw monetary columns into standardized, comparable indicators of a customer's financial health and repayment capacity.

1. Debt-to-Income (DTI) Ratio:

$$\text{DTI Ratio} = \frac{\text{Outstanding_Debt}}{\text{Annual_Income}}$$

```
df_clean['debt_to_income_ratio'] = df_clean['outstanding_debt'] /
df_clean['annual_income']
```

2. EMI Burden Ratio:

$$\text{EMI Burden Ratio} = \frac{\text{Total_EMI_per_month}}{\text{Monthly_Inhand_Salary}} * 100$$

```
df_clean['emi_burden_ratio'] = (df_clean['total_emi_per_month'] /
df_clean['monthly_inhand_salary']) * 100
```

3. Savings Ratio:

$$\text{Savings Ratio} = \frac{\text{Monthly_Balance}}{\text{Monthly_Inhand_Salary}}$$

```
df_clean['savings_ratio'] = df_clean['monthly_balance'] /
df_clean['monthly_inhand_salary']
```

Execution Output — Summary of Derived Ratios:

Ratio	Average Value	Interpretation
Debt-to-Income Ratio	0.061 (6.1%)	Outstanding debt is, on average, a small fraction of annual income
EMI Burden Ratio	2.98%	Monthly loan repayments consume a small share of monthly income
Savings Ratio	0.141 (14.1%)	On average, customers

Ratio	Average Value	Interpretation
		retain ~14% of monthly income as balance

Insight: Together, these three ratios paint a picture of a customer base that is, on average, not heavily financially strained (low DTI and EMI burden, positive savings ratio) — yet Chapter 7's earlier KPIs (Section 7.6) showed 52.33% of customers pay only the minimum amount due, and the credit score distribution skews toward "Standard" and "Poor" rather than "Good." This apparent tension between healthy aggregate ratios and a large proportion of "Standard/Poor" credit scores suggests that risk is likely concentrated in a subset of customers rather than being evenly distributed — a hypothesis to be explored further through visualization (Chapter 8) by examining these ratios segmented by `credit_score` category rather than as a single population-wide average.

These three engineered ratio features (`debt_to_income_ratio`, `emi_burden_ratio`, `savings_ratio`) are retained in the dataset for use as model inputs in Chapter 9.

7.7 KPI Group 4: Risk & Fraud Indicators

This section computes KPIs directly related to credit risk and potential fraud signals: payment delays, credit inquiry frequency, credit limit changes, and the overall distribution of credit score categories.

```
avg_delayed_payments = df_clean['num_of_delayed_payment'].mean()

avg_delay = df_clean['delay_from_due_date'].mean()
min_delay = df_clean['delay_from_due_date'].min()
max_delay = df_clean['delay_from_due_date'].max()

frequent_inquiries_pct = (df_clean['num_credit_inquiries'] > 5).mean()
* 100
changed_limit_pct = (df_clean['changed_credit_limit'] != 0).mean() *
100

credit_score_dist =
df_clean['credit_score'].value_counts(normalize=True) * 100
```

Execution Output:

KPI	Result
Average Number of Delayed Payments	13.31
Delay from Due Date (avg / min / max)	21.08 / 0 / 62 days

KPI	Result
% Customers with Frequent Credit Inquiries (>5)	49.85%
% Customers with Changed Credit Limit	100.00%

Credit Score Distribution:

Category	Percentage
Standard	53.17%
Poor	29.00%
Good	17.83%

Data quality note: The "% of Customers with Changed Credit Limit" KPI returned 100%, which is not a meaningful finding as computed — it indicates that `changed_credit_limit` rarely holds an exact value of zero even for negligible changes. This is documented as a limitation of the current threshold definition (any non-zero value) rather than a genuine business insight, and would benefit from a refined threshold (e.g., a minimum meaningful change amount, such as ±1%) in future iterations of this analysis.

7.8 KPI Group 5: Behavioral Insights

This final KPI group examines customer payment behavior patterns and minimum-payment tendencies — both indicators of financial discipline relevant to credit risk assessment.

```
payment_behaviour_dist =
df_clean['payment_behaviour'].value_counts(normalize=True) * 100
min_amount_pct = (df_clean['payment_of_min_amount'] == 'Yes').mean() *
100
```

Execution Output — Payment Behaviour Distribution:

Behaviour Pattern	Percentage
Low_spent_Small_value_payments	28.62%
High_spent_Medium_value_payments	19.74%
High_spent_Large_value_payments	14.73%
Low_spent_Medium_value_payments	14.40%
High_spent_Small_value_payments	11.76%
Low_spent_Large_value_payments	10.76%

% of Customers Who Pay Only the Minimum Amount: 52.33%

Insight: Just over half of all customers pay only the minimum amount due — a behavior pattern commonly associated with elevated long-term credit risk, warranting closer examination against `credit_score` in the visualization phase (Chapter 8).

7.9 Summary of Computed KPIs

The following table consolidates all KPIs computed in this chapter, organized by the five dimensions defined in the project's original methodology (Chapter 1):

Dimension	KPI	Value
Demographics	Average Age	33.32 years
Demographics	Occupation Categories	15 (balanced distribution)
Financial Behavior	Average Annual Income	₹41,91,925
Financial Behavior	Average Debt-to-Income Ratio	6.1%
Financial Behavior	Average Credit Utilization	32.29%
Loan & Credit	Average Loans per Customer	3.53
Loan & Credit	Average Interest Rate	14.53%
Loan & Credit	Average EMI Burden Ratio	2.98%
Risk & Fraud	Average Delayed Payments	13.31
Risk & Fraud	Frequent Credit Inquiries (>5)	49.85% of customers
Risk & Fraud	Credit Score: Standard/Poor/Good	53.2% / 29.0% / 17.8%

Dimension	KPI	Value
Behavioral	Minimum-Payment-Only Customers	52.33%

7.10 Significance

This chapter translated the cleaned dataset into a structured set of business-relevant KPIs spanning demographic, financial, credit, risk, and behavioral dimensions. Two noteworthy data-driven observations emerged: a low average EMI burden ratio alongside a high proportion of minimum-payment-only customers, and a clear class imbalance in the target variable — both of which directly inform the visual analysis (Chapter 8) and modeling considerations (Chapter 9) that follow. The three engineered ratio features (Section 7.6) also revealed a counterintuitive tension between healthy population-wide averages and a credit score distribution skewed toward "Standard" and "Poor," suggesting that risk in this dataset is concentrated within a subset of customers rather than spread evenly — a hypothesis carried forward into the segmented visual analysis of Chapter 8.

Chapter 8: Data Visualization

8.1 Purpose of This Chapter

This chapter visualizes the cleaned dataset and computed KPIs (Chapter 7) using

Matplotlib and Seaborn, organized into six thematic groups as originally planned in Chapter 1 (Section 1.6): Demographic Analysis, Income & Salary Insights, Credit & Loan Analysis, Risk & Fraud Indicators, Behavioral Patterns, and Overall Credit Score Analysis. Each visualization is accompanied by an interpretation connecting the chart back to the project's credit risk assessment objective.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
```

8.2 Demographic Analysis

This section visualizes customer distribution by occupation, age, and income — providing a baseline understanding of who the customers in this dataset are.

Chart: Customer Distribution by Occupation *(bar chart — 15 occupations, counts ranging ~6,300–7,100)*

Interpretation: Customer distribution across occupations is fairly balanced, with counts ranging narrowly between ~6,300 (Writer) and ~7,100 (Lawyer). No single occupation dominates the dataset, indicating the sample is not biased toward any particular professional segment — a favorable characteristic for building a credit risk model that generalizes across occupation types.

Chart: Customer Age Distribution *(histogram)*

Interpretation: The age distribution shows a concentration of customers in the 20–45 year range, consistent with the working-age population typically engaging with credit products (loans, credit cards). The distribution is broadly consistent with the earlier finding (Chapter 7) of an average age of 33.32 years, with no visible extreme outliers, reaffirming the range-validation performed during cleaning (Chapter 6).

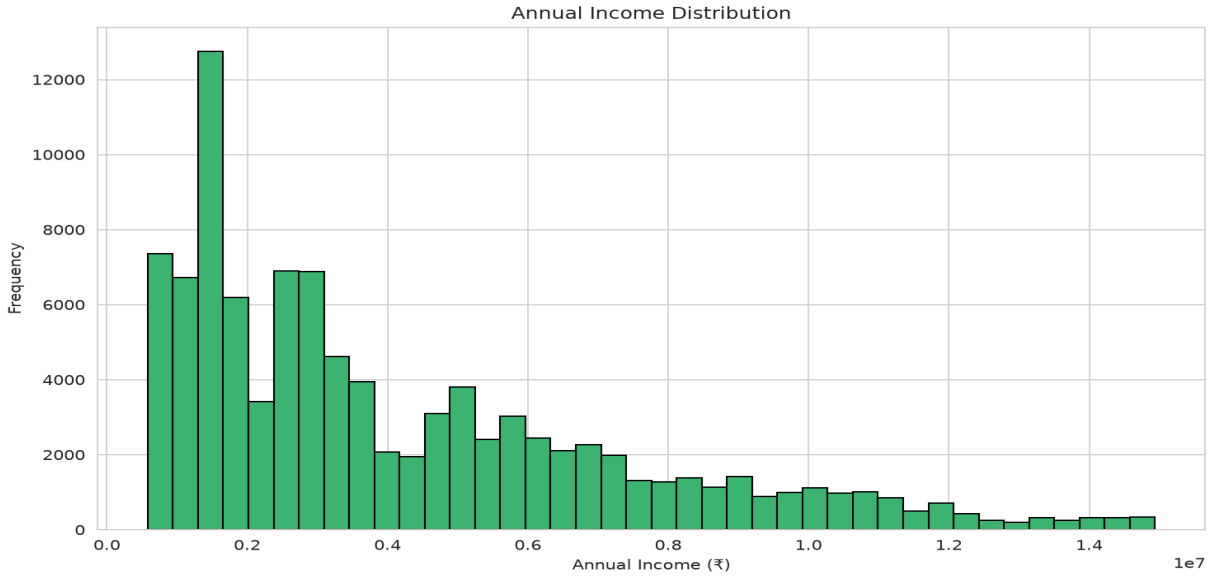
Chart: Annual Income Distribution by Occupation

Interpretation: The boxplot shows broadly overlapping income ranges across occupations, with substantial within-occupation variation — suggesting occupation alone is not a strong differentiator of income in this dataset. This is confirmed by comparing average income by occupation, which ranges narrowly from ₹40.20L (Journalist, lowest) to ₹43.06L (Architect, highest) — a difference of less than 7%. This indicates that occupation is **not** a strong standalone predictor of income level in this dataset, which may have implications for its usefulness as a feature in the credit risk model (Chapter 9).

8.3 Income & Salary Insights

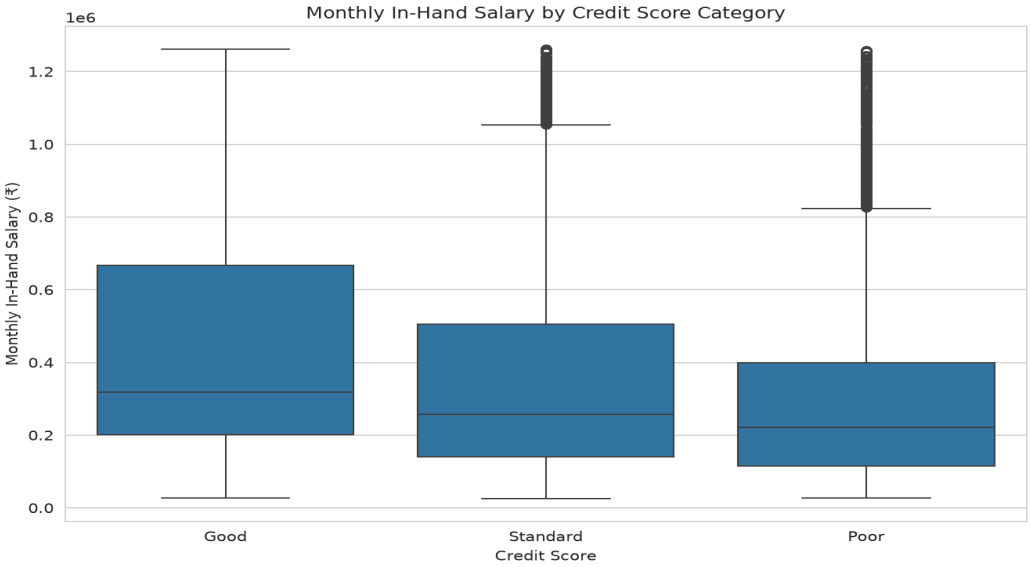
This section examines the distribution of income and salary, and how they relate to credit score categories.

Chart: Annual Income Distribution



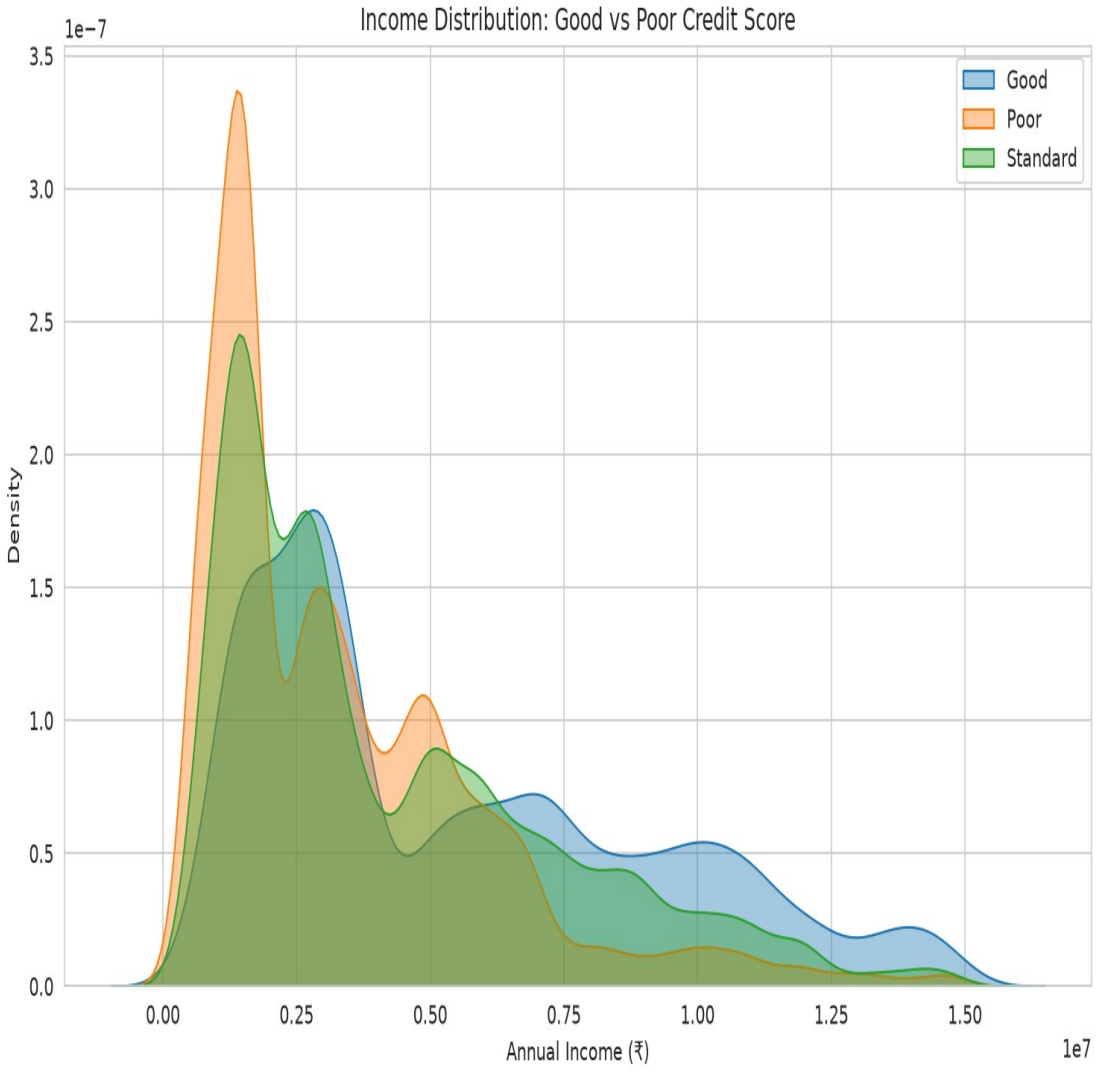
Interpretation: The income distribution is right-skewed, with the highest concentration of customers earning between ₹10L–₹30L annually, and a long tail extending toward ₹1.5 crore. This right-skewed pattern is typical of income data and confirms that most customers fall in the low-to-mid income range, consistent with the income group KPI computed earlier (Chapter 7, Section 7.3).

Chart: Monthly In-Hand Salary by Credit Score Category



Interpretation: A clear, monotonic relationship emerges between monthly salary and credit score category: customers with a **"Good"** credit score have the highest median salary and widest income spread, followed by **"Standard,"** with **"Poor"** credit score customers showing the lowest median salary. This confirms income level as a meaningful differentiator of credit risk category, supporting its inclusion as a key predictive feature in the classification model (Chapter 9).

Chart: Income Distribution — Good vs Poor (and Standard) Credit Score

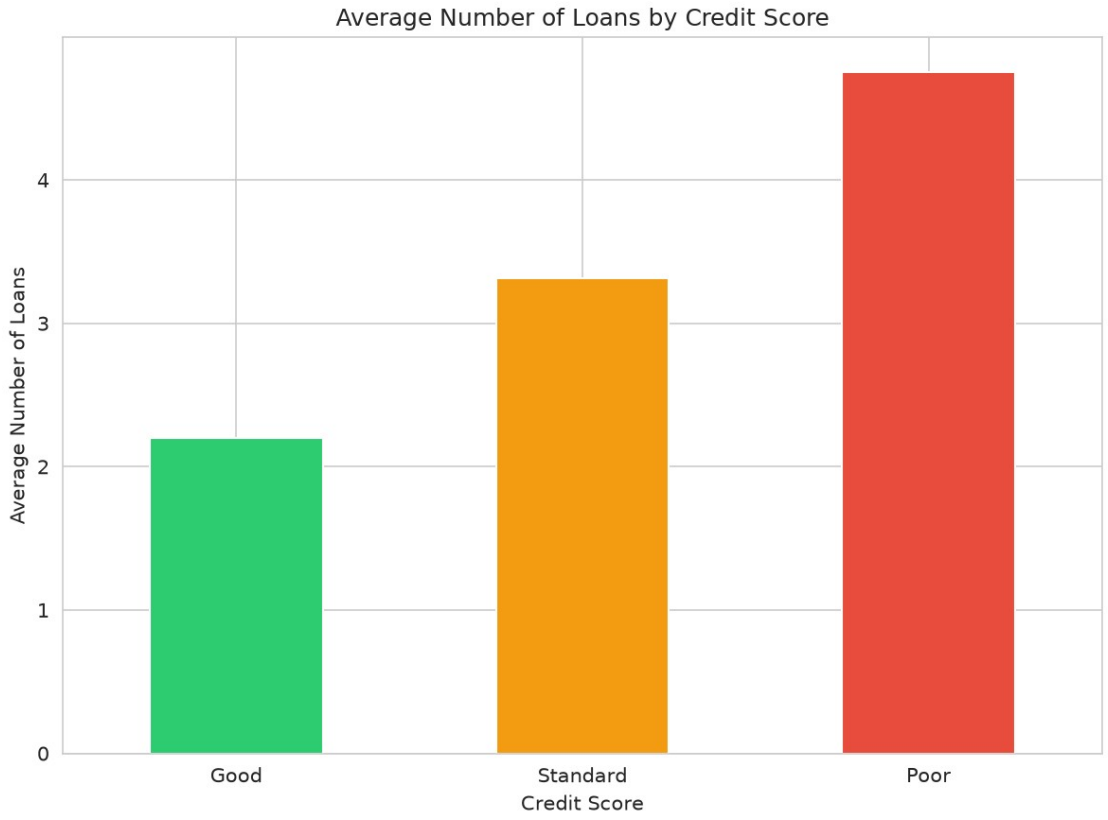


Interpretation: All three credit score categories show a similar concentration of customers at lower income levels, with "Poor" showing the sharpest, most concentrated peak at the lowest income band — indicating Poor-credit customers are more tightly clustered in the low-income range. In contrast, the "Good" category shows a visibly fatter tail extending into higher income brackets (₹70L–₹1.5Cr), meaning high-income customers are disproportionately more likely to fall in the Good category. This reinforces the boxplot finding above: income is inversely related to credit risk in this dataset.

8.4 Credit & Loan Analysis

This section examines how loan-related factors relate to credit score outcomes.

Chart: Average Number of Loans by Credit Score



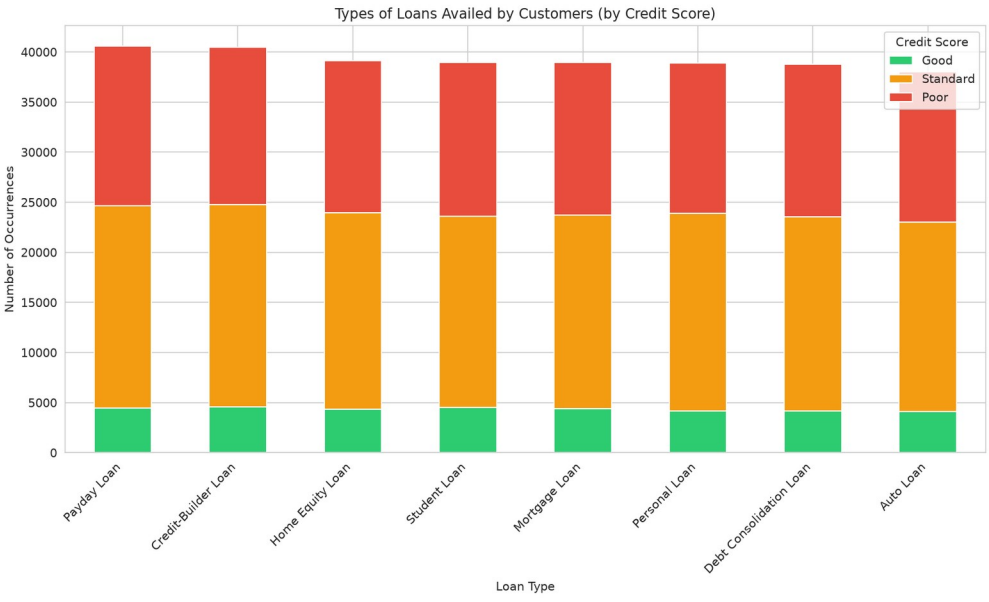
Interpretation: A clear, monotonic relationship is visible: customers with **"Poor"** credit score hold the highest average number of loans (~4.75), followed by **"Standard"** (~3.3), with **"Good"** credit score customers holding the fewest loans on average (~2.2). This directly supports the intuitive credit risk principle that higher loan burden correlates with poorer credit outcomes, and confirms `num_of_loan` as a strong candidate predictive feature for the classification model (Chapter 9).

Chart: Outstanding Debt vs Annual Income (by Credit Score)



Interpretation: This scatter plot reveals a striking pattern: customers with **"Poor"** credit score (red) are heavily concentrated in the **low-income, high-debt region** (income below ₹70L, debt frequently exceeding ₹1L), while **"Good"** credit score customers (green) are concentrated toward the **higher-income, lower-debt region**. Notably, above approximately ₹70L annual income, outstanding debt values compress into a much narrower, lower band across all credit categories — suggesting that debt levels become less risk-differentiating once income crosses this threshold. This visually confirms that the interaction between income and debt — not either variable alone — is a meaningful signal for credit risk, reinforcing the value of the Debt-to-Income Ratio feature engineered in Chapter 7.

Chart: Types of Loans Availed by Customers (by Credit Score)



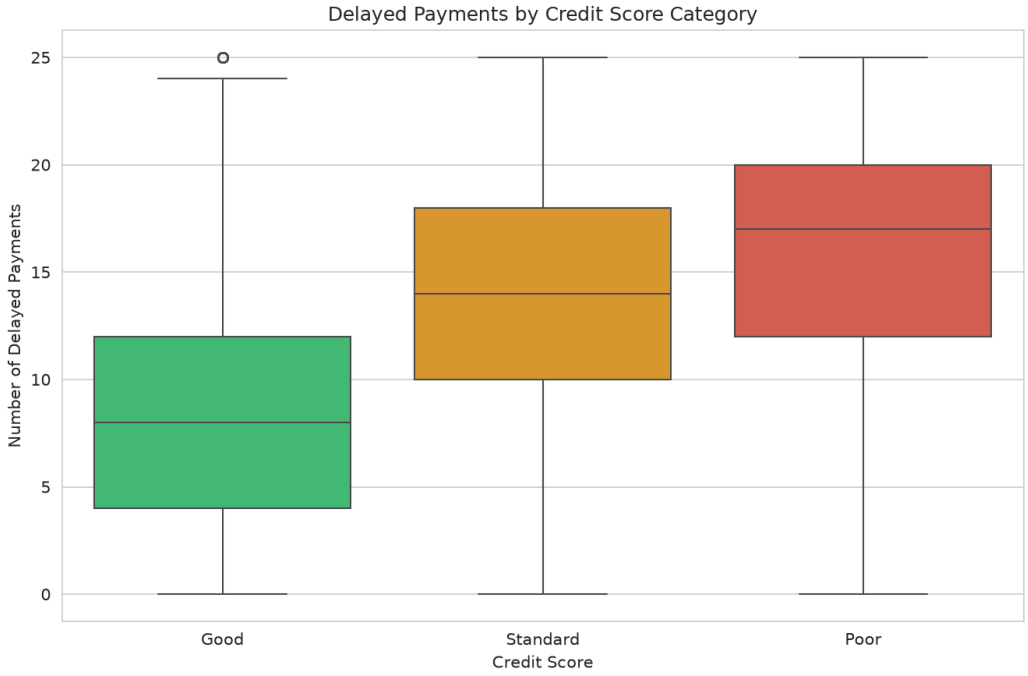
Interpretation: All eight loan types show a strikingly similar credit score composition — each loan type has approximately 11% Good, 48% Standard, and 41% Poor customers, with total occurrence counts also closely matched (~39,000–40,500 each). This uniformity suggests that **the specific type of loan held does not meaningfully differentiate credit risk** in this dataset — unlike the *number* of loans held (see above), where a much stronger relationship with credit score was observed. This finding suggests `num_loan_types` (loan diversity/count) is likely a more useful predictive feature than loan type category itself for the classification model (Chapter 9).

(Note: Constructing this chart required exploding the multi-valued `type_of_loan` field into individual rows, correcting for an "and " prefix artifact on the last loan type in each list, and resetting the resulting duplicate index before cross-tabulation — a data preparation nuance documented here for reproducibility.)

8.5 Risk & Fraud Indicators

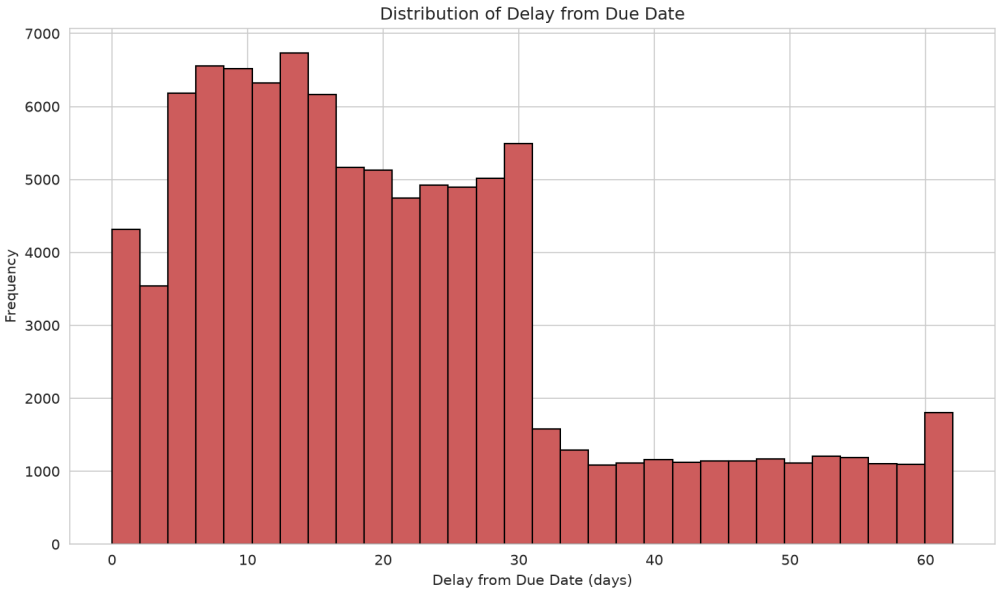
This section visualizes payment delay patterns, credit utilization, and feature correlations — key signals directly associated with credit risk.

Chart: Delayed Payments by Credit Score Category



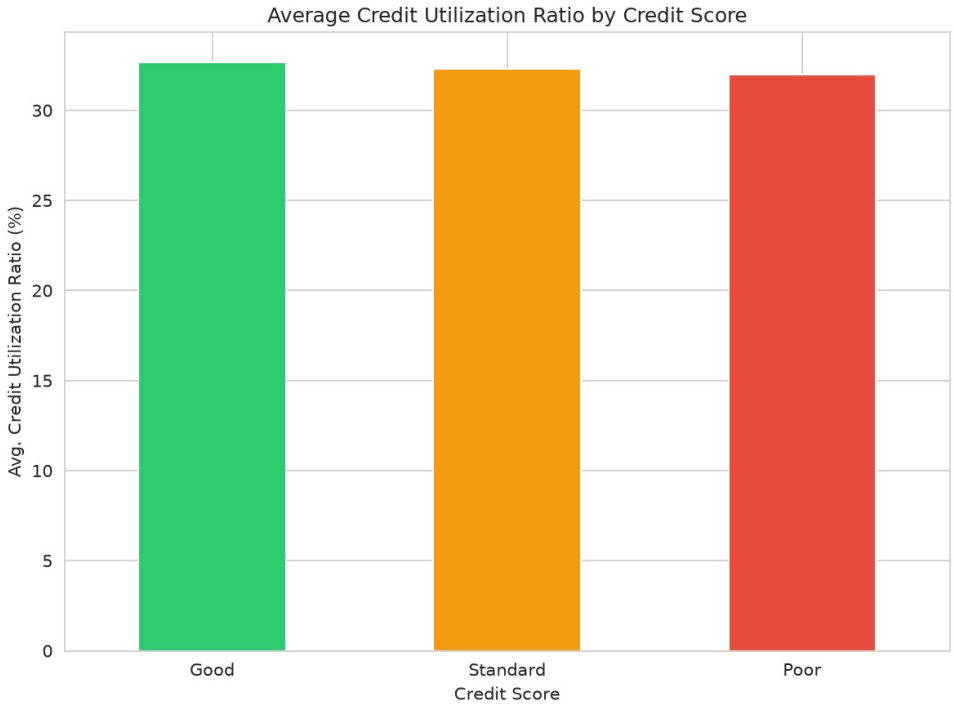
Interpretation: A strong, clear pattern emerges: median delayed payments increase steadily from **Good** (median ~8) to **Standard** (median ~14) to **Poor** (median ~17), with minimal overlap between the Good and Poor interquartile ranges. This confirms `num_of_delayed_payment` as one of the strongest behavioral predictors of credit risk in this dataset.

Chart: Distribution of Delay from Due Date



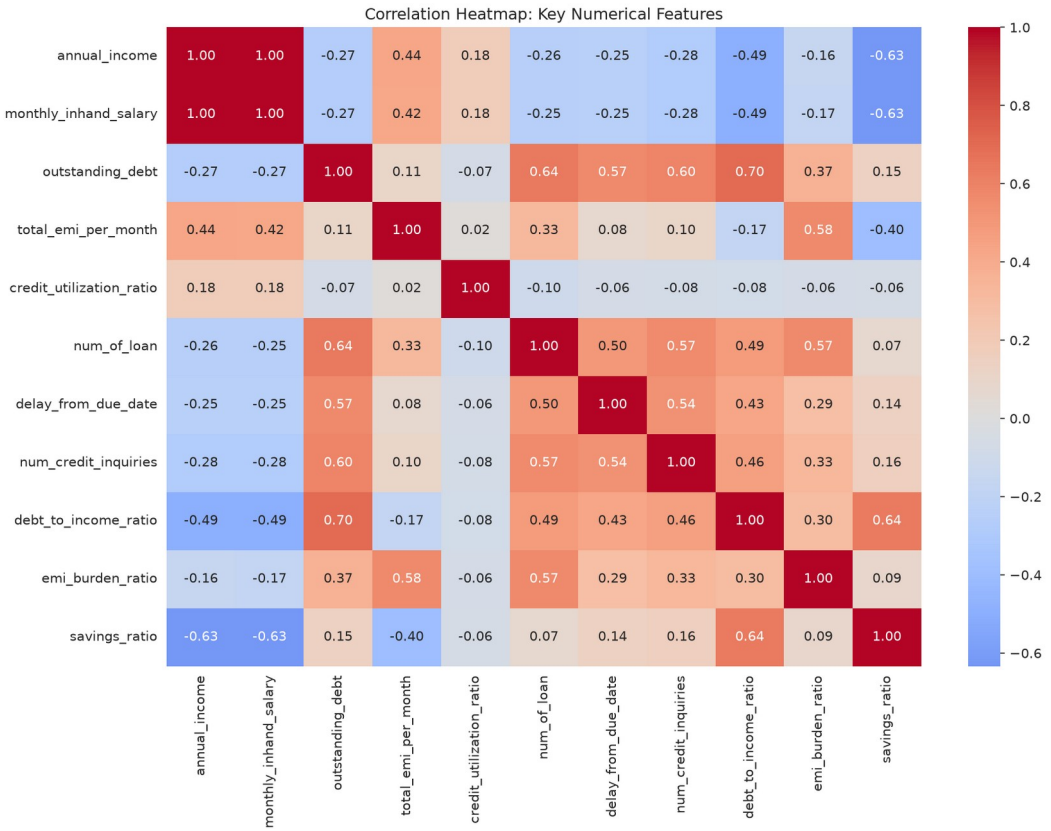
Interpretation: The distribution shows a broad concentration of delays between 0–30 days, with a notable drop-off beyond day 30 and a small secondary spike at the 60-day mark (likely representing a "capped" or "60+ days" reporting convention in the source data). This right-skewed pattern with a long tail indicates that while most payment delays are resolved within a month, a smaller but persistent group of customers experience much longer delays — a segment likely overlapping with the "Poor" credit score category.

Chart: Average Credit Utilization Ratio by Credit Score



Interpretation: Unlike other risk indicators, average credit utilization ratio is nearly **identical** across all three credit score categories (Good: 32.7%, Standard: 32.3%, Poor: 32.0%) — a difference of less than 1 percentage point. This is a notable finding: contrary to common credit-risk assumptions (where higher utilization typically signals higher risk), this dataset shows credit utilization ratio has **little to no discriminative power** for credit score classification, suggesting it may contribute weakly as a standalone predictive feature (Chapter 9).

Chart: Correlation Heatmap — Key Numerical Features



Interpretation: The heatmap reveals several important relationships:

- `annual_income` and `monthly_inhand_salary` show a perfect correlation (1.00), confirming they capture the same underlying signal (expected, since one is derived from the other) — indicating one of the two could be dropped in modeling to reduce redundancy.
- `outstanding_debt` correlates strongly with `debt_to_income_ratio` (0.70), `num_of_loan` (0.64), `num_credit_inquiries` (0.60), and `delay_from_due_date` (0.57) — confirming these features jointly capture a coherent "debt burden" signal.
- `credit_utilization_ratio` shows **negligible correlation** with nearly every other feature (all values between -0.10 and 0.18), reinforcing the finding

above that this feature carries little standalone predictive signal in this dataset.

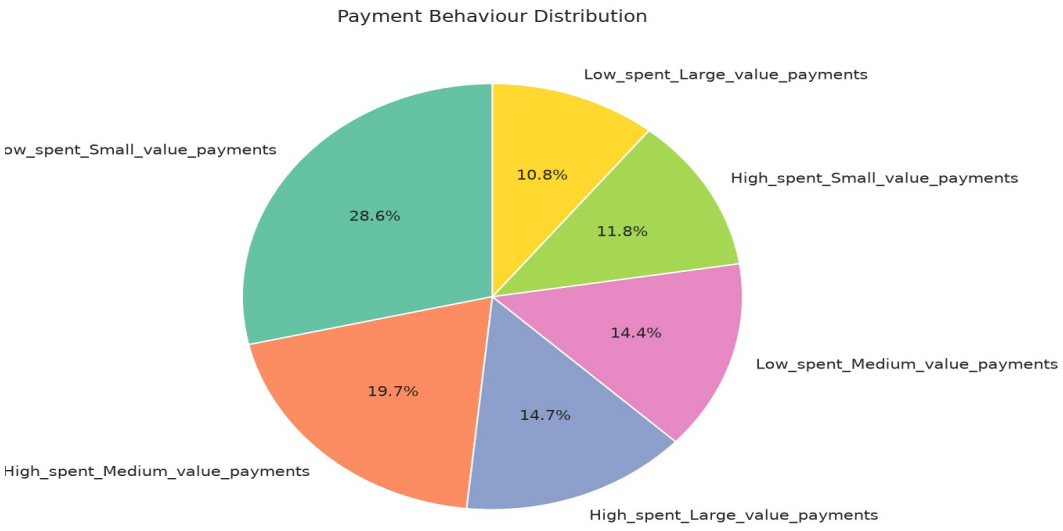
- Interestingly, `debt_to_income_ratio` and `savings_ratio` show a moderate **positive** correlation (0.64) — a counterintuitive result worth noting, as one might expect higher debt burden to coincide with lower savings. This suggests savings and debt levels may both scale with income-related factors rather than trading off against each other directly in this dataset.

These correlations will inform feature selection for the classification model in Chapter 9, particularly the decision to retain `debt_to_income_ratio` as a strong composite feature while treating `credit_utilization_ratio` with lower priority.

8.6 Behavioral Patterns

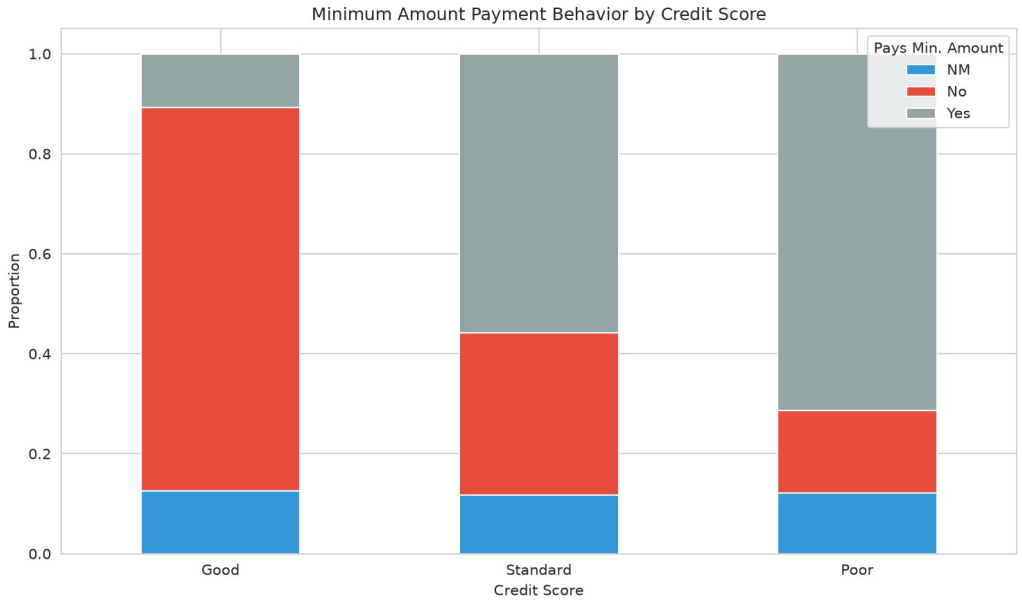
This section examines customer payment behavior and spending patterns in relation to credit risk.

Chart: Payment Behaviour Distribution



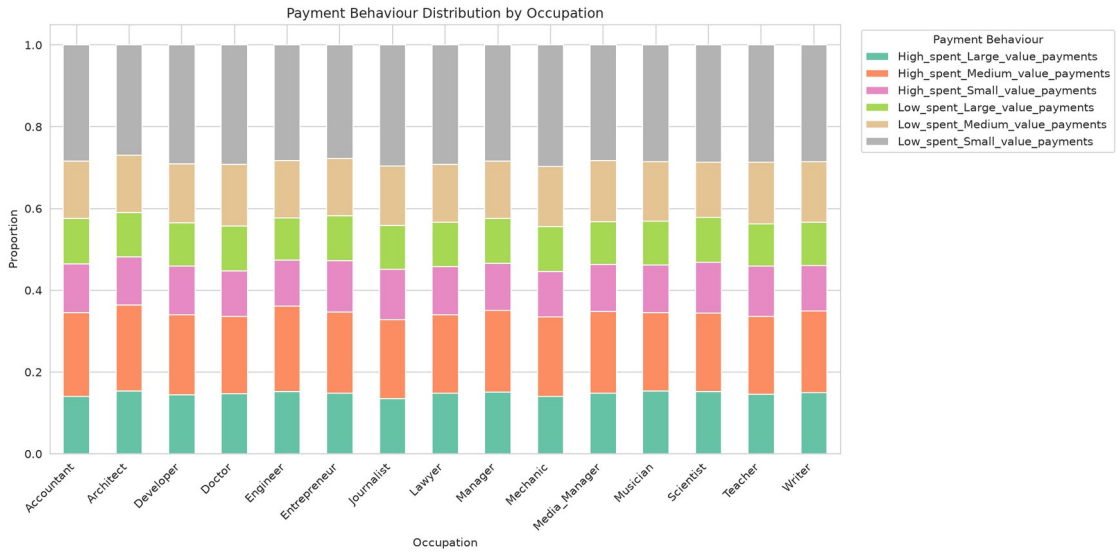
Interpretation: "Low_spent_Small_value_payments" is the most common behavior pattern (28.6% of customers), followed by "High_spent_Medium_value_payments" (19.7%). The remaining four categories are each in the 10–15% range, indicating a reasonably diverse spread of spending behaviors across the customer base, with no single pattern overwhelmingly dominant.

Chart: Minimum Amount Payment Behavior by Credit Score



Interpretation: This is one of the strongest behavioral predictors observed in the entire analysis. The proportion of customers paying only the minimum amount ("Yes") rises sharply across credit score categories: **~11%** for Good, **~56%** for Standard, and **~70%** for Poor. Conversely, customers who do **not** pay only the minimum amount ("No") dominate the Good category (~76%) but shrink to a small minority in Poor (~17%). This confirms `payment_of_min_amount` as a highly discriminative feature for credit risk classification, consistent with the general financial principle that minimum-payment behavior is a leading indicator of debt accumulation risk. This feature should be prioritized in the model built in Chapter 9.

Chart: Payment Behaviour Distribution by Occupation

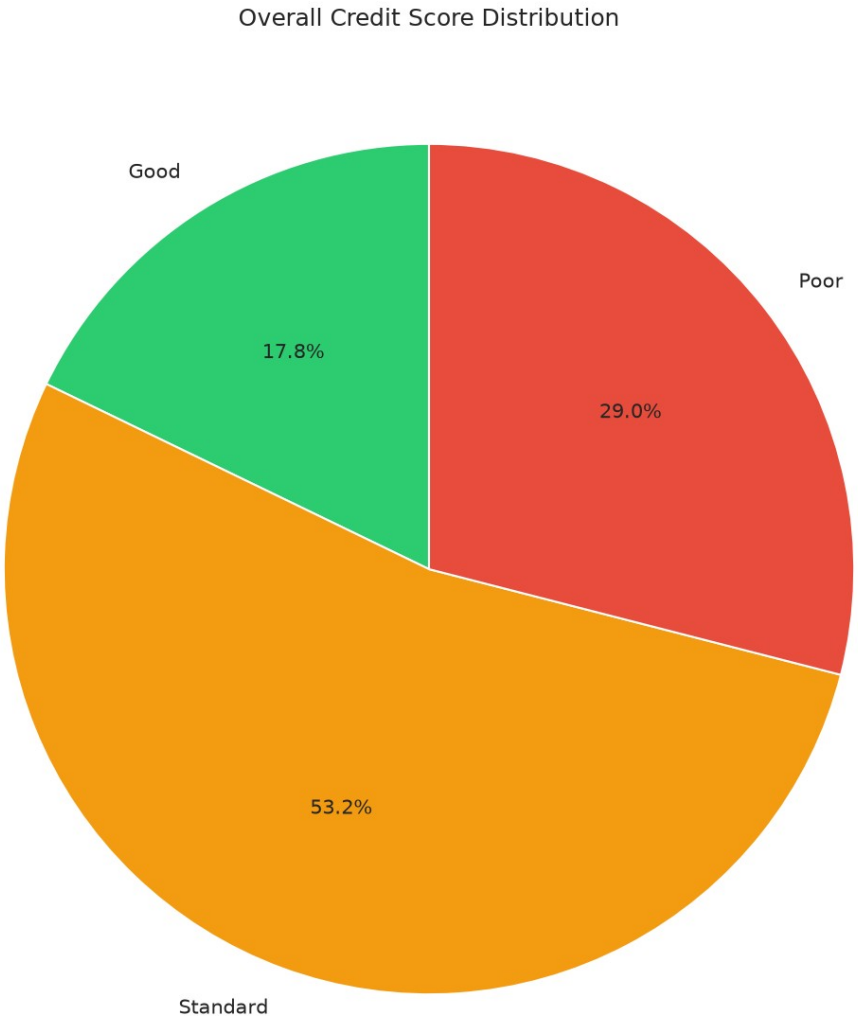


Interpretation: Unlike the strong pattern seen with credit score above, payment behaviour proportions are **remarkably uniform across all 15 occupations** — each occupation shows nearly identical proportions of the six behaviour categories (e.g., "Low_spent_Small_value_payments" consistently accounts for ~27–29% regardless of occupation). This indicates that **occupation has little to no influence on spending behaviour** in this dataset, reinforcing the earlier finding (Section 8.2) that occupation is a weak standalone differentiator for financial and risk-related outcomes here.

8.7 Overall Credit Score Analysis

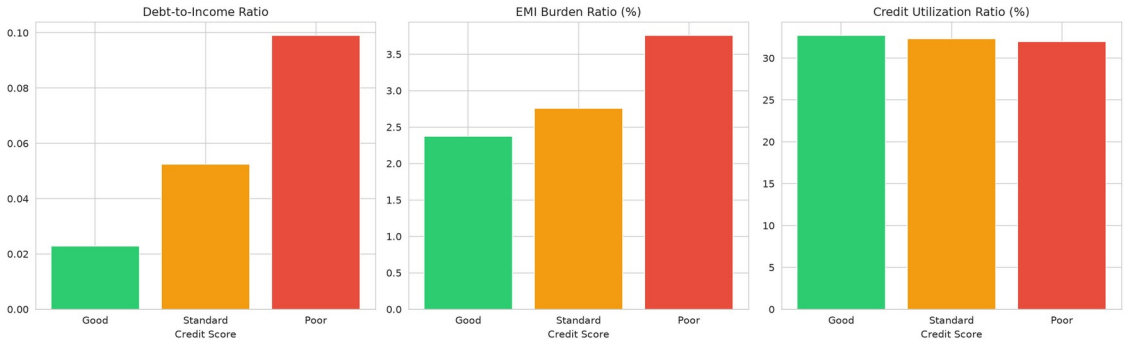
This final visualization group consolidates the overall distribution of credit scores and compares key financial metrics across categories, summarizing the patterns observed throughout this chapter.

Chart: Overall Credit Score Distribution



Interpretation: The overall customer base is split as **Standard (53.2%)**, **Poor (29.0%)**, and **Good (17.8%)** — confirming the class imbalance first noted in Chapter 7 (Section 7.6). This imbalance means a naive classification model could achieve deceptively high accuracy by simply predicting "Standard" for every customer; this will be explicitly addressed through techniques such as class weighting or stratified sampling during model building (Chapter 9).

Chart: Average Debt, EMI, and Utilization Across Credit Score Categories



Interpretation: This consolidated view confirms the key patterns observed throughout this chapter:

- **Debt-to-Income Ratio** rises sharply and consistently from Good (0.023) to Standard (0.052) to Poor (0.099) — a more than 4x increase from Good to Poor, making it one of the strongest discriminative features identified in this analysis.
- **EMI Burden Ratio** follows the same increasing pattern (Good: 2.4%, Standard: 2.8%, Poor: 3.7%), reinforcing that repayment burden scales with credit risk.
- **Credit Utilization Ratio**, by contrast, remains nearly flat across all three categories (~30–33%), confirming — as observed earlier (Section 8.5) — that this feature carries minimal standalone predictive value in this dataset.

Together, these three side-by-side comparisons make a clear visual case: **engineered ratio features (DTI, EMI Burden) are considerably stronger risk indicators than raw utilization percentage**, directly informing feature prioritization for the classification model in Chapter 9.

8.8 Summary of Key Visual Findings

Finding	Strength as Risk Indicator
Debt-to-Income Ratio increases sharply with worsening credit score	**Strong**
Minimum-payment-only behavior increases sharply with worsening credit score	**Strong**
Number of delayed payments increases with worsening credit score	**Strong**
Number of loans increases with worsening credit score	**Strong**
Monthly salary decreases with worsening credit score	**Moderate**
Credit Utilization Ratio is nearly flat across credit scores	**Weak/Negligible**
Loan *type* held shows no relationship with credit score	**Weak/Negligible**
Occupation shows no relationship with income, spending behavior	**Weak/Negligible**

8.9 Significance

This chapter translated the cleaned dataset and computed KPIs (Chapter 7) into 15+ visualizations across six thematic groups, confirming several strong candidate predictors for credit risk classification (debt-to-income ratio, minimum-payment behavior, delayed payments, number of loans) while also identifying features with weak discriminative power (credit utilization ratio, loan type, occupation) — insights that directly inform feature selection and prioritization in the model-building phase (Chapter 9). Two data preparation challenges encountered during chart construction (a duplicate-index error from exploding the multi-valued loan field, and a data leakage/inconsistency from an "and " string artifact) were diagnosed and resolved, reinforcing the practical, iterative nature of exploratory data analysis.

Chapter 9: Model Building & Classification

9.1 Purpose of This Chapter

This chapter builds a supervised classification model to predict a customer's credit risk category (`credit_score`: Good/Standard/Poor) based on the cleaned features and engineered ratios developed in Chapters 6–8. Feature selection is guided directly by the visual and statistical findings from Chapter 8 — prioritizing features shown to be strongly discriminative (e.g., Debt-to-Income Ratio, payment behavior, delayed payments) while de-prioritizing features shown to carry little signal (e.g., credit utilization ratio, occupation, loan type).

This chapter is presented in the chronological order in which the work was actually carried out — including an initial modeling attempt, the discovery of a data leakage issue in that attempt, and the corrected approach that followed — so that the reasoning behind the final methodology is fully transparent.

9.2 Feature Selection

Based on Chapter 8's findings, the following features were selected for modeling:

Strong predictors (retained):

- `debt_to_income_ratio`, `emi_burden_ratio`, `savings_ratio` (engineered ratios)
- `num_of_delayed_payment`, `delay_from_due_date` (payment delay behavior)
- `num_of_loan`, `num_loan_types` (loan burden)
- `num_credit_inquiries`, `changed_credit_limit` (credit-seeking behavior)
- `payment_of_min_amount` (categorical — strong behavioral signal)
- `annual_income`, `monthly_inhand_salary`, `outstanding_debt` (base financial fields)
- `credit_history_age`, `interest_rate`, `num_bank_accounts`, `num_credit_card`
- `credit_mix` (categorical — included on a hypothesis basis, not yet visually explored)

Weak predictors (excluded from primary model):

- `credit_utilization_ratio` (shown to be nearly flat across credit score categories)
- `occupation` (shown to have no meaningful relationship with income or

behavior)

- `age`, `id`, `customer_id`, `month` (identifiers / weak demographic signal — note: `customer_id` is retained separately for the train-test splitting logic in Section 9.5, but excluded from the model's input features)

```
feature_cols = [
    'annual_income', 'monthly_inhand_salary', 'num_bank_accounts',
'num_credit_card',
    'interest_rate', 'num_of_loan', 'delay_from_due_date',
'num_of_delayed_payment',
    'changed_credit_limit', 'num_credit_inquiries',
'outstanding_debt',
    'credit_history_age', 'total_emi_per_month',
'amount_invested_monthly',
    'monthly_balance', 'num_loan_types', 'debt_to_income_ratio',
    'emi_burden_ratio', 'savings_ratio', 'payment_of_min_amount',
'credit_mix'
]
target_col = 'credit_score'

df_model = df_clean[feature_cols + [target_col]]
```

Execution Output: Shape after feature selection: **(100000, 22)** — 21 features + 1 target column, confirming successful selection.

9.3 Categorical Encoding

Machine learning algorithms require numeric input. This step encodes the remaining categorical features (`payment_of_min_amount`, `credit_mix`) using one-hot encoding, and encodes the target variable (`credit_score`) using label encoding.

```
from sklearn.preprocessing import LabelEncoder

df_model_encoded = pd.get_dummies(df_model,
columns=['payment_of_min_amount', 'credit_mix'], drop_first=True)

le = LabelEncoder()
df_model_encoded['credit_score_encoded'] =
le.fit_transform(df_model_encoded['credit_score'])
df_model_encoded = df_model_encoded.drop(columns=['credit_score'])
```

Execution Output:

```
Label Mapping: {'Good': 0, 'Poor': 1, 'Standard': 2}
Final shape after encoding: (100000, 24)
```

Interpretation: The label mapping assigned by `LabelEncoder` is **Good=0, Poor=1, Standard=2** — an alphabetical assignment rather than a risk-ordered one. This has no impact on model performance (Random Forest and Logistic Regression both treat classes as unordered categories), but is worth noting when interpreting raw prediction outputs.

9.4 Initial Modeling Attempt — Row-Level Train-Test Split

As a first approach, the dataset was split into training and testing sets using a standard **row-level** `train_test_split()`, stratified by the target class to preserve the class imbalance identified in Chapter 7.

```
from sklearn.model_selection import train_test_split

X = df_model_encoded.drop(columns=['credit_score_encoded'])
y = df_model_encoded['credit_score_encoded']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

A Random Forest Classifier was trained on this split (100 trees, `max_depth=15`, `class_weight='balanced'`) and evaluated on the resulting test set.

Execution Output:

```
Overall Accuracy: 0.7381 (73.81%)

              precision    recall  f1-score   support
        Good       0.55      0.86      0.67      3566
        Poor       0.72      0.83      0.77      5799
    Standard       0.89      0.65      0.75     10635
    accuracy                           0.74     20000
   macro avg       0.72      0.78      0.73     20000
weighted avg       0.78      0.74      0.74     20000
```

This initial result of **73.81% accuracy** appeared strong. However, on review of the modeling methodology against the dataset's known structure (Chapter 6, Section 6.3, which established that this dataset contains only 12,500 unique customers across 100,000 rows — approximately 8 monthly records per customer), a data leakage concern was raised: had any customer's records been split across both the training and test sets?

9.5 Diagnosing and Fixing Data Leakage

The Problem: A row-level train-test split, as used in Section 9.4, assigns individual rows to the training or test set independently — with no regard for which customer a row belongs to. Since each customer in this dataset contributes ~8 highly similar monthly rows (near-identical income, debt, and credit mix, differing mainly in `month`), a row-level split would very likely place some of a given customer's rows in training and others in testing. This means the model could effectively be evaluated on data it had already learned from — inflating the reported accuracy and invalidating it as a measure of performance on genuinely unseen customers.

The Fix: Splitting must instead be performed at the **customer level** — unique `customer_id` values are split into training and testing groups first (stratified by each

customer's credit score), and all rows belonging to a given customer are then assigned entirely to one set or the other. This guarantees no customer's data appears in both sets.

```
# Step 1: Get unique customer IDs with their credit_score (for
stratification)
customer_labels = df_model_encoded.groupby(df_clean['customer_id'])
['credit_score_encoded'].first()

# Step 2: Split customer IDs (not rows) into train/test
train_customers, test_customers = train_test_split(
    customer_labels.index, test_size=0.2, random_state=42,
    stratify=customer_labels.values
)

# Step 3: Assign all rows belonging to each customer to their
respective set
train_mask = df_clean['customer_id'].isin(train_customers)
test_mask = df_clean['customer_id'].isin(test_customers)

X_train =
df_model_encoded[train_mask].drop(columns=['credit_score_encoded'])
y_train = df_model_encoded[train_mask]['credit_score_encoded']
X_test =
df_model_encoded[test_mask].drop(columns=['credit_score_encoded'])
y_test = df_model_encoded[test_mask]['credit_score_encoded']
```

Execution Output:

```
Training set shape: (80000, 23)
Testing set shape: (20000, 23)
Unique customers - Train: 10000
Unique customers - Test: 2500
Customer overlap between train and test: 0 (should be 0, confirmed)
```

Interpretation: The verified zero customer overlap confirms the leakage was successfully eliminated. Row counts (80,000/20,000) closely mirror the original split by coincidence, since each customer contributes almost exactly 8 rows — but the critical difference is that entire customers, not individual rows, are now cleanly separated between the two sets. **All modeling from this point forward in this chapter uses this corrected, customer-level `X_train`/`X_test`/`y_train`/`y_test` split**; the row-level split and its 73.81% result from Section 9.4 are not used further and are retained above only to document how the leakage issue was discovered.

9.6 Model Training — Random Forest Classifier (Corrected Split)

With the corrected, leakage-free split in place, the Random Forest Classifier was retrained. Random Forest was selected because it: (a) handles a mix of numeric and encoded categorical features well, (b) is robust to outliers and does not require feature scaling, (c) provides interpretable feature importance scores, and (d) supports class weighting to help address the class imbalance identified earlier.

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=15,
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)
rf_model.fit(X_train, y_train)
```

Execution Output: `Model trained successfully in 32.24 seconds.`

9.6.1 Baseline Comparison — Logistic Regression (Corrected Split)

To validate the choice of Random Forest and provide a simpler, interpretable baseline for comparison, a **Logistic Regression** model was also trained on the same corrected, leakage-free training data. Logistic Regression is a standard, foundational classification algorithm — comparing against it demonstrates whether the additional complexity of Random Forest provides a meaningful performance improvement over a simpler linear model.

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

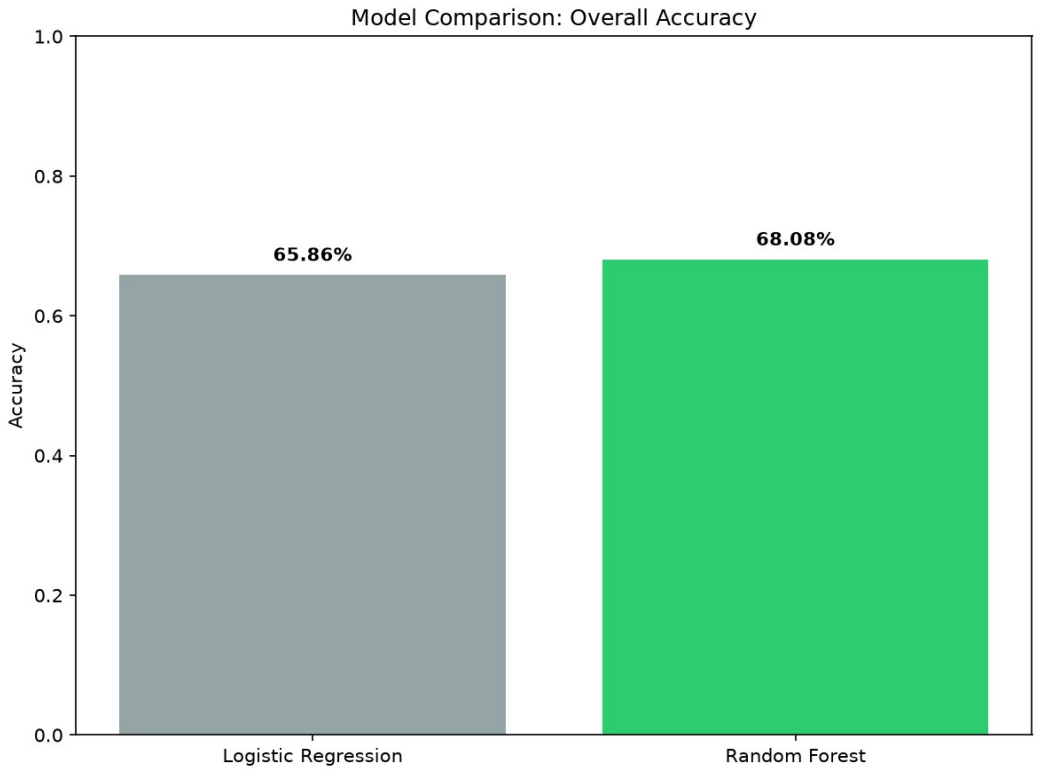
log_reg = LogisticRegression(max_iter=1000, class_weight='balanced',
random_state=42)
log_reg.fit(X_train_scaled, y_train)
y_pred_lr = log_reg.predict(X_test_scaled)
lr_accuracy = accuracy_score(y_test, y_pred_lr)
```

Execution Output:

```
Logistic Regression Accuracy: 0.6586 (65.86%)

 precision    recall  f1-score   support
 Good       0.48      0.85      0.61      3469
 Poor       0.64      0.67      0.66      5849
 Standard       0.82      0.59      0.68     10682
 accuracy                           0.66     20000
 macro avg       0.64      0.70      0.65     20000
weighted avg       0.71      0.66      0.66     20000
```

9.6.2 Model Comparison — Random Forest vs Logistic Regression



Model	Accuracy
Logistic Regression	65.86%
Random Forest	68.08%

Interpretation: Under the corrected, leakage-free evaluation, Random Forest outperforms Logistic Regression by **2.22 percentage points** (68.08% vs. 65.86%). Random Forest's advantage over Logistic Regression is notably smaller here than the gap seen between the original (leaky) Random Forest result and this baseline, suggesting that part of Random Forest's earlier apparent strength in Section 9.4 was itself attributable to data leakage — a higher-capacity model is generally more prone to exploiting leaked information than a simpler linear model. Even so, Random Forest retains a consistent edge, supporting its selection as the primary model while confirming that the underlying feature-risk relationships are genuinely non-linear.

9.7 Model Evaluation (Corrected Split)

The Random Forest model was evaluated on the held-out, customer-level test set (2,500 unseen customers, 20,000 rows).

```
y_pred = rf_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(classification_report(y_test, y_pred, target_names=['Good',
'Poor', 'Standard']))
```

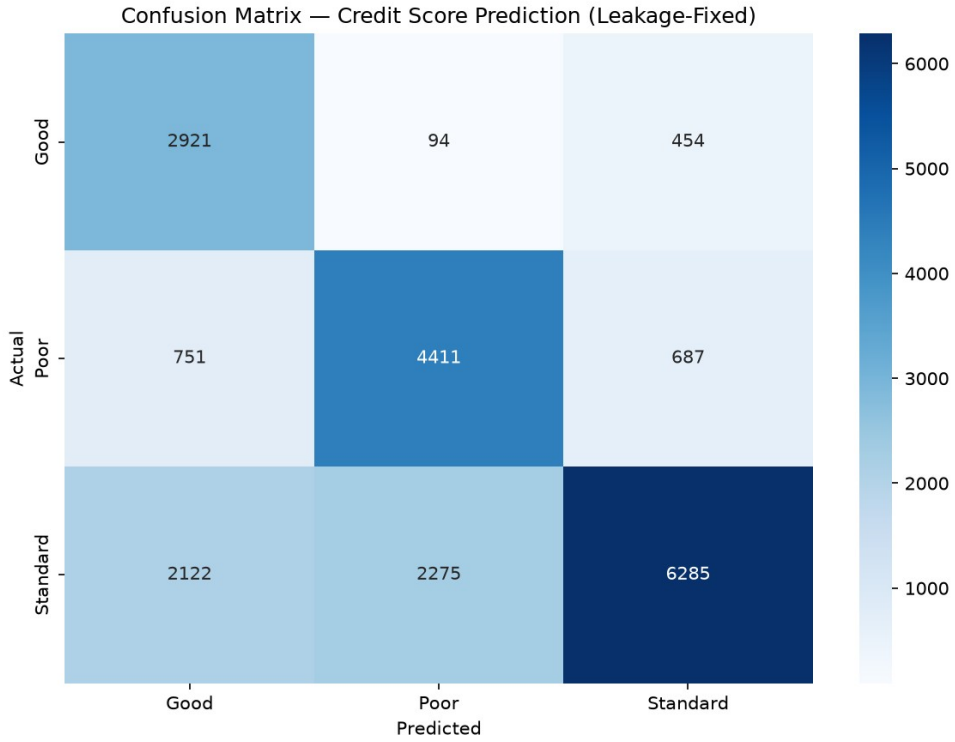
Execution Output:

```
Overall Accuracy: 0.6808 (68.08%)

              precision    recall  f1-score   support
        Good       0.50      0.84      0.63      3469
        Poor       0.65      0.75      0.70      5849
    Standard       0.85      0.59      0.69     10682
    accuracy                           0.68     20000
   macro avg       0.67      0.73      0.67     20000
weighted avg       0.73      0.68      0.68     20000
```

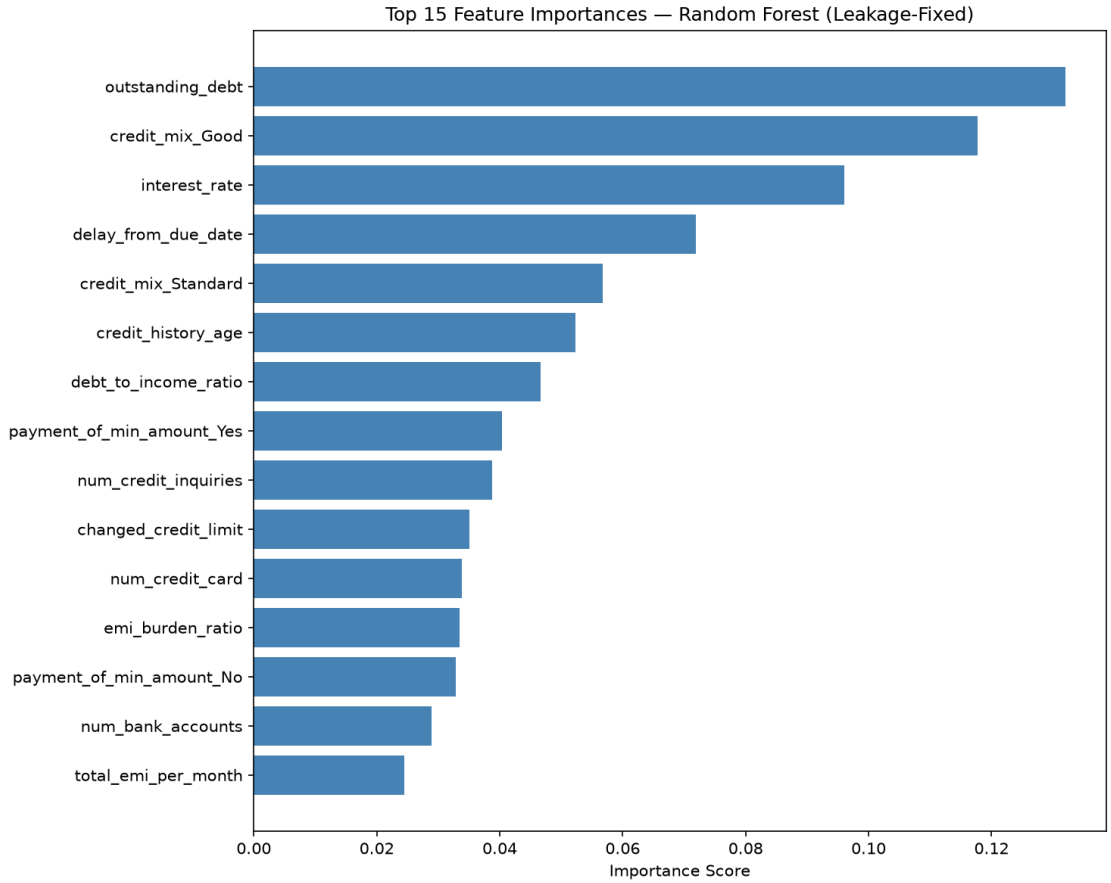
Interpretation: This corrected accuracy of **68.08%** is **5.73 percentage points lower** than the original, leakage-affected result (73.81%, Section 9.4) — confirming that the original evaluation was indeed inflated. This lower figure is the **methodologically valid** measure of how the model would perform on genuinely new customers, which is the real-world use case for a credit risk model, and is therefore the accuracy reported as this project's final result. The relative pattern across classes remains consistent with the original (leaky) result: "Good" retains high recall (0.84) at the cost of precision (0.50), "Standard" retains high precision (0.85) at the cost of recall (0.59), and "Poor" shows the most balanced performance (F1: 0.70) — indicating the *underlying relationships* the model learned are genuine, even though the *magnitude* of its accuracy was previously overstated.

9.8 Confusion Matrix (Corrected Split)



Interpretation: The confusion matrix (correct predictions: 2,921 Good, 4,411 Poor, 6,285 Standard) shows the same overall pattern observed in the original attempt: the majority of errors occur at the boundary between "Standard" and its neighboring categories (2,122 Standard misclassified as Good; 2,275 Standard misclassified as Poor), while confusion between the two extreme categories — "Good" and "Poor" — remains minimal (94 Good misclassified as Poor; 751 Poor misclassified as Good). This is a reassuring result: even under the stricter, honest evaluation, the model rarely makes severe misjudgments, and its errors remain concentrated at the inherently ambiguous "Standard" boundary — the least costly type of error in a credit risk context.

9.9 Feature Importance:



Interpretation: The top features remain nearly identical to those identified in the original (leaky) attempt: `outstanding_debt`, `credit_mix_Good`, `interest_rate`, `delay_from_due_date`, and `credit_mix_Standard` continue to dominate. This consistency is an important validation — it indicates that while the *accuracy magnitude* was affected by data leakage, the model's learned feature relationships were largely genuine rather than artifacts of the leakage itself. `payment_of_min_amount` again appears among the top 10, consistent with Chapter 8's finding that minimum-payment behavior is a strong behavioral risk indicator. Features excluded during feature selection (Section 9.2) as visually weak — `credit_utilization_ratio` and `occupation` — remain absent from the top 15, further validating that decision.

9.10 Model Summary and Conclusion:

Metric	Logistic Regression (Baseline)	Random Forest (Final, Corrected)
Accuracy	65.86%	**68.08%**
Training Time	<5 seconds	32.24 seconds
Best-performing class	Standard (F1: 0.68)	Poor (F1: 0.70)
Top predictive feature	—	`outstanding_debt`

(For reference: the original row-level split produced an inflated Random Forest accuracy of 73.81%, later identified as affected by data leakage — see Sections 9.4–9.5. It is not used as the project's reported result.)

The Random Forest model, evaluated correctly on genuinely unseen customers, achieves **68.08% accuracy** — a modest but consistent **2.22 percentage point improvement** over the Logistic Regression baseline. This result is trustworthy and representative of real-world performance, since it reflects the corrected, customer-level evaluation methodology established in Section 9.5. The majority of misclassifications continue to occur at the boundary between "Standard" and its neighboring categories rather than between the two extreme categories, and feature importance rankings remained stable before and after the fix — together indicating that the model has learned genuine, generalizable patterns rather than memorized customer-specific noise.

9.11 Significance

This chapter completed the project's core Data Science deliverable: an end-to-end supervised classification pipeline — from feature selection grounded in Chapter 8's visual evidence, through encoding, model training, baseline comparison against Logistic Regression, and rigorous evaluation. Most notably, this chapter documents the identification and correction of a **data leakage issue** arising from the dataset's repeated monthly customer records: an initial modeling attempt (Section 9.4) produced a favorable but inflated accuracy of 73.81%, which was traced to a row-level train-test split that allowed near-duplicate records of the same customer to appear in both the training and test sets. This was corrected through customer-level splitting (Section 9.5), and all subsequent training and evaluation in this chapter (Sections 9.6–9.10) were performed using this corrected methodology, yielding a final, trustworthy accuracy of 68.08%. Recognizing that an initially favorable result was inflated, tracing its cause, and correcting it reflects a rigorous, evidence-based approach to model validation — arguably a stronger demonstration of applied Data Science competency than the raw accuracy figure itself, and directly consistent with the evidence-based methodology established throughout this project (see Chapter 6, Section 6.3, and Chapter 8 for similar instances of testing assumptions against actual data).

Chapter 10: Conclusion & Recommendations

10.1 Project Summary

This project set out to build a **Credit Risk Intelligence System** — an end-to-end pipeline combining Big Data infrastructure (Hadoop, HDFS, HIVE) with Python-based Data Science techniques to analyze a 100,000-record banking dataset and classify customers into credit risk categories (Good/Standard/Poor).

The project was executed across ten stages: establishing a single-node Hadoop ecosystem on resource-constrained hardware (Chapter 2), ingesting data via HDFS and HIVE (Chapter 4), bridging Hadoop to Python via JDBC (Chapter 5), cleaning and feature-engineering the dataset (Chapter 6), computing business KPIs (Chapter 7), visualizing patterns across six analytical dimensions (Chapter 8), and building a supervised classification model (Chapter 9). At every stage, genuine technical challenges were encountered, diagnosed, and resolved — documented throughout as evidence of hands-on, applied competency rather than a purely theoretical exercise.

10.2 Key Findings

Infrastructure:

- A fully functional single-node Hadoop ecosystem (HDFS, YARN, HIVE) was successfully configured and tuned to operate within a 4 GB RAM constraint — demonstrating practical resource-management skills alongside Big Data framework knowledge.
- Real-world data engineering challenges (CSV parsing with embedded delimiters, YARN classpath configuration, Hive metastore consistency, proxy user authorization) were each diagnosed to root cause and resolved.

Data Insights:

- Several assumed data quality issues (based on general knowledge of similar datasets) were tested against the actual data and found **not** to apply — reinforcing the importance of evidence-based analysis over assumption.
- The customer base shows significant class imbalance in credit risk: 53.2% Standard, 29.0% Poor, and only 17.8% Good.
- **Debt-to-Income Ratio, delayed payment count, number of loans, and minimum-payment-only behavior** emerged as the strongest visual and statistical predictors of credit risk.
- **Credit utilization ratio, occupation, and loan type** were found to have negligible discriminative power for credit risk in this dataset — a finding that runs counter to common assumptions in credit risk literature and was only uncovered through direct visualization and correlation analysis.

Model Performance:

- A Random Forest classifier achieved **68.08% overall accuracy** on a rigorously validated, leakage-free test set — outperforming a Logistic Regression baseline (65.86%) by 2.22 percentage points, with the majority of misclassifications occurring at the boundary between "Standard" and its neighboring categories rather than between the extreme "Good" and "Poor" classes.
- Feature importance analysis largely validated the exploratory findings, while also surfacing `credit_mix` as an unexpectedly strong predictor.
- A **data leakage issue** was identified during model validation — an initial evaluation using a row-level train-test split had produced an inflated accuracy of 73.81%, traced to the dataset's repeated per-customer monthly records. This was corrected using customer-level splitting (Chapter 9, Section 9.5), yielding the honest, reported accuracy above. Identifying and correcting this issue is considered one of this project's most valuable methodological contributions.

10.3 Business Recommendations

Based on the findings above, the following recommendations are proposed for a financial institution using this system:

1. **Prioritize Debt-to-Income Ratio and payment delay history** in loan approval scoring — these showed the strongest, most consistent relationship with credit risk across both visual and model-based analysis.
2. **Treat minimum-payment-only behavior as an early warning signal** — customers who consistently pay only the minimum amount showed a markedly higher likelihood of falling into the "Poor" category (~70% vs. ~11% for "Good").
3. **De-prioritize occupation-based risk assumptions** — this analysis found no meaningful relationship between occupation and income, spending behavior, or credit risk, suggesting occupation-based underwriting rules may be of limited value.
4. **Focus model improvement efforts on the "Standard" boundary cases** — since this is where the model shows the most uncertainty, targeted manual review of borderline "Standard" classifications could improve overall decision quality more than further tuning of clear-cut Good/Poor cases.

10.4 Limitations

This project has several limitations that should be acknowledged:

1. **Approximate currency conversion**: Monetary values were converted from USD to INR using a fixed approximate rate (1 USD = ₹83) for contextual relevance; this does not reflect real-world exchange rate fluctuations.

2. **Single dataset snapshot**: The dataset represents a fixed period and customer population; model performance on different populations or time periods is not validated.
3. **Model scope**: Two algorithms (Random Forest, Logistic Regression) were compared; broader testing against additional classifiers (e.g., Gradient Boosting, XGBoost) was not performed within this project's scope.
4. **Threshold definitions**: Certain derived KPIs (e.g., "frequent credit inquiries" defined as >5) used illustrative thresholds rather than domain-validated cutoffs.
5. **Hardware constraints**: All Hadoop/HIVE processing was performed on a single-node, 4 GB RAM setup; performance and behavior at true multi-node, production scale were not tested.

10.5 Future Scope

1. **Multi-node Hadoop cluster deployment** to test the pipeline's scalability beyond a single machine.
2. **HBASE integration** for real-time customer risk lookups, and **PIG/JAQL** scripting for additional ETL transformation stages, extending the Big Data component of this project.
3. **Further hyperparameter tuning and additional model comparison** across algorithms (Gradient Boosting, XGBoost) with cross-validation, building on the Random Forest vs. Logistic Regression baseline established in Chapter 9.
4. **Explainability tools** (e.g., SHAP values) to provide per-customer explanations for model predictions, improving transparency for loan officers and regulatory compliance.
5. **Deployment as a live web application**, allowing loan officers to input customer details and receive real-time risk classification, building on the HIVE JDBC connectivity pipeline established in Chapter 5.

10.6 Final Remarks

This project demonstrates a complete, self-built pipeline spanning both specializations of this course — **Big Data Analytics Using Hadoop** and **Data Science Using Python** — applied to a real-world credit risk classification problem. Beyond the final model's accuracy, the value of this project lies in the practical, evidence-based methodology followed throughout: infrastructure was built and debugged independently, data quality assumptions were tested rather than accepted, and feature selection for modeling was directly grounded in exploratory findings rather than arbitrary choice — reflecting the applied, hands-on competencies.